BPP-00 — Build Projection Pack

BPP-00 — Build Projection Pack

1. Purpose

Define the Build Projection Pack as the Agent Kit layer that translates rich documentation into build-executable structure.

This pack exists to solve the exact gap you identified:

- the Agent Kit knows a lot about the app
- but the build system cannot materialize all of that knowledge unless it is converted into repo-facing projections

So the Build Projection Pack is the layer that compiles the Agent Kit into:

- module truth
- repo truth
- file truth
- implementation-unit truth

Without this pack, the builder keeps collapsing a rich kit into a shallow scaffold.

---

2. Core role

The Build Projection Pack is the **build-compilation layer** inside the Agent Kit.

It sits between:

- descriptive artifacts
  and
- repo generation

In flow terms:

Requirements / Design / Architecture / Data / API / Security / Acceptance
→ Build Projection Pack
→ Repo Blueprint
→ Build Plan
→ Repo Generation

That is its role.

---

3. Main goal

The main goal of the Build Projection Pack is to convert abstract or document-heavy kit knowledge into explicit build targets that a generator can follow with minimal guessing.

In practical terms, it should answer:

- what modules exist
- what pages/screens exist
- what endpoints exist
- what entities/schemas exist
- what tests/checks exist
- what files should exist
- how requirements map into implementation units

That is what makes the kit materially buildable.

---

4. Why this pack is needed

A normal Agent Kit can be strong in:

- requirements
- product logic

- design
- architecture
- security
- API
- data
- acceptance

But still weak for generation because those truths remain mostly descriptive.

The Build Projection Pack is needed because the builder does not generate from "understanding the product" alone.
It generates from:

- module maps
- inventories
- contracts
- file targets
- trace maps

So this pack is what turns the Agent Kit into a true build authority.

---

5. Core principle

The Build Projection Pack must not invent new product truth.
It must **project existing approved kit truth into executable build structures**.

That means it should:

- compile
- decompose

- map
- enumerate
- trace

But not:

- redefine scope
- add major features
- silently broaden the architecture
- guess new modules outside approved truth

This keeps the pack aligned with the Agent Kit rather than competing with it.

---

6. What this pack produces

At minimum, the Build Projection Pack should produce these projection artifact classes:

- module inventory
- UI/screen inventory
- endpoint inventory
- entity/schema inventory
- file inventory
- requirement trace map
- verification trace map
- repo blueprint seed

These are the projection artifacts that make the rest of Build Mode possible.

---

7. What this pack consumes

The Build Projection Pack should consume the richer descriptive layers of the Agent Kit, including:

- build brief
- canonical spec
- work breakdown
- acceptance map
- architecture pack
- implementation pack
- design pack
- data pack
- API pack
- security pack
- ops/governance pack where relevant

So the pack is not an independent design step.
It is a compiler layer over already-approved kit truth.

---

8. What this pack is not

The Build Projection Pack is not:

- the Repo Blueprint itself
- the Build Plan itself
- the generator

- the verifier
- a separate product-planning system
- a replacement for architecture or requirements docs

It is a translation layer between Agent Kit knowledge and buildable structure.

---

9. System position

The Build Projection Pack should sit **inside the Agent Kit branch**, not inside late-stage repo generation.

That matters because if you wait until Build Mode generation to derive all projections, the generator has to improvise too much.

So the correct sequence is:

1. Agent Kit is assembled
2. Build Projection Pack is derived from the Agent Kit
3. Repo Blueprint is derived from the Build Projection Pack
4. Build Plan is derived from the Repo Blueprint
5. Repo is generated

That is the right order.

---

10. Required projection outputs

The Build Projection Pack should guarantee that the build system can derive, at minimum:

10.1 Structural build truth

- repo shape
- module boundaries

- subsystem responsibilities
- target layers

10.2 Implementation-unit truth

- services
- pages/screens
- components
- routes/endpoints
- schemas/entities
- tests/proof targets

10.3 Materialization truth

- expected file classes
- expected file inventory
- requirement-to-file traceability

These are the minimum things needed to stop build collapse.

---

11. Minimum internal sections

The Build Projection Pack should be organized into a small number of focused artifacts, not one giant dump.

Minimum sections:

1. module projection
2. interface projection
3. data projection

4. UI projection
5. file projection
6. trace projection
7. repo projection seed

That structure keeps it usable.

---

12. Build authority relationship

The Build Projection Pack should strengthen the Agent Kit's authority, not dilute it.

The correct relationship is:

- Agent Kit = approved product/build truth
- Build Projection Pack = executable decomposition of that truth
- Repo Blueprint = repository structure derived from that decomposition
- Build Plan = execution path derived from that blueprint

So the pack is the bridge that lets the Agent Kit remain central all the way through repo generation.

---

13. Anti-drift function

This pack is one of the most important anti-drift layers in the whole system.

Why?

Because without it, the generator must perform hidden translation like:

- requirement prose → module guess
- architecture intent → service guess

- design description → page guess
- data/security rules → backend guess

That hidden translation is where most repo collapse happens.

The Build Projection Pack removes much of that ambiguity by making implementation targets explicit before generation starts.

---

14. Completion condition

The Build Projection Pack should only be considered complete when the Agent Kit has been projected into enough explicit structural and implementation units that a Repo Blueprint can be derived without major generator guessing.

If it cannot do that, the pack is incomplete.

---

15. Success definition

BPP-00 is successful when it establishes the Build Projection Pack as the mandatory translation layer that compiles Agent Kit truth into executable build structures—module inventories, interface inventories, schema inventories, file inventories, and trace maps—so the internal build system can remain truly kit-driven instead of reducing the kit to a shallow scaffold.

---

16. Short version for the doc

BPP-00 — Build Projection Pack
The Build Projection Pack is the Agent Kit layer that converts descriptive kit knowledge into executable build structure. It compiles approved requirements, architecture, design, data, API, security, and acceptance truth into explicit module inventories, interface inventories, schema inventories, file inventories, and trace maps. Its purpose is to make the Agent Kit materially buildable and to prevent the internal build system from collapsing a rich kit into a shallow scaffold.

BPP-01 — Projection Scope

BPP-01 — Projection Scope

1. Purpose

Define exactly what kinds of build-facing structures the Build Projection Pack must project out of the Agent Kit.

BPP-01 exists so the Build Projection Pack does not become vague. It should make clear what source knowledge must be translated, what target projection classes must be produced, and where the projection layer stops.

---

2. Core rule

The Build Projection Pack must project **all build-relevant kit truth**, not just feature names or high-level summaries.

That means the scope of projection includes everything in the Agent Kit that materially affects:

- repo shape
- subsystem design
- implementation units
- interfaces
- data structures
- security behavior
- verification obligations
- file inventory

If the builder needs it to generate the repo correctly, it belongs in projection scope.

---

3. Main scope principle

The main principle of BPP-01 is:

Every approved kit artifact that changes what the repo must contain, how it must be structured, or what it must prove should be projected into an executable build-facing form.

That is the line that defines projection scope.

---

4. Source scope

The Build Projection Pack should consume and project from these Agent Kit source layers.

4.1 Build Brief

Used for:

- app identity
- app type
- high-level priorities
- stack orientation
- output expectations

4.2 Canonical Spec

Used for:

- normalized product truth
- core entities and concepts
- primary app boundaries
- resolved assumptions

4.3 Work Breakdown

Used for:

- work units

- feature decomposition
- implementation grouping
- delivery structure

4.4 Acceptance Map

Used for:

- build obligations
- proof obligations
- test targets
- completion expectations

4.5 Architecture Pack

Used for:

- subsystems
- module boundaries
- layer structure
- service boundaries
- dependency direction

4.6 Implementation Pack

Used for:

- implementation units
- technical components
- service expectations

- technical flows
- execution details

4.7 Design Pack

Used for:

- pages
- layouts
- views
- component families
- interaction states

4.8 Data Pack

Used for:

- entities
- schemas
- relationships
- migrations
- storage concerns

4.9 API Pack

Used for:

- route inventory
- endpoint contracts
- request/response surfaces

- integration points
- events/webhooks if relevant

4.10 Security Pack

Used for:

- auth model
- RBAC/permission surfaces
- security middleware obligations
- data protection requirements
- guarded flows

4.11 Ops / Governance Pack

Used where relevant for:

- required operational artifacts
- audit obligations
- config expectations
- policy-linked build requirements

---

5. Projection target scope

The Build Projection Pack must project these source layers into explicit build-facing targets.

5.1 Module projections

What modules/services/subsystems exist.

5.2 Interface projections

What routes/endpoints/events/contracts exist.

5.3 Data projections

What entities/schemas/migrations/storage structures exist.

5.4 UI projections

What screens/pages/layouts/components/states exist.

5.5 File projections

What directories, file classes, and concrete file targets exist.

5.6 Trace projections

How requirements and acceptance obligations map to modules/files/tests.

5.7 Repo projections

What overall repo shape and structural layout are implied.

That is the minimum target scope.

---

6. Structural projection scope

BPP-01 should explicitly include projection of repo-level structure.

That means the Build Projection Pack must project:

- repo type
- repo shape
- layer layout
- subsystem placement
- module placement
- shared/common areas
- config/docs/test/supporting areas where required

This is important because repo collapse often happens when the system projects features but not structure.

---

7. Implementation-unit projection scope

BPP-01 should explicitly include projection of implementation units.

That means the pack must project things like:

- services
- handlers
- routes
- models
- hooks
- pages
- UI components
- forms
- validators
- tests
- scripts
- middleware
- background jobs if required
- integration adapters if required

If the unit is real enough that it should become code, it belongs in scope.

---

8. Contract projection scope

The Build Projection Pack must also project contract-bearing surfaces.

That includes:

- API contracts
- shared type surfaces
- schema contracts
- state/flow contracts
- permission surfaces
- test/proof obligations

This matters because many repos fail not from missing features, but from missing interface and contract structure.

---

9. Verification projection scope

BPP-01 must include projection of build-verification targets.

That means it must identify:

- what must be tested
- what must be proven
- what files or modules carry acceptance obligations
- what artifacts are needed to satisfy completion logic

Without this, acceptance remains descriptive instead of build-driving.

---

10. File-volume projection scope

This section should explicitly include **file volume projection**.

That means the pack must project:

- expected directory count
- expected module count
- expected file classes
- expected concrete file inventory or file-target inventory
- expected file-count range or explicit expected file count

This is critical for solving the underbuilt-repo problem.

If file volume is not in projection scope, the build system will keep compressing the repo.

---

11. What is inside projection scope but often missed

These are the areas many systems under-project and you should explicitly include them in scope:

- middleware/security surfaces
- audit/logging surfaces
- background jobs
- integration adapters
- migrations and seed files
- validation layers
- permission checks
- shared contracts/types
- configuration files
- support scripts
- proof/test files

This is where build systems often lose enterprise seriousness.

---

12. What is outside projection scope

The Build Projection Pack should not project things that are not build-relevant or are too far downstream.

Outside scope:

- deployment runtime health
- production infrastructure provisioning
- live environment state
- post-export human development changes
- business decisions not already approved in the kit
- speculative future features

So the pack is broad, but still bounded.

---

13. Scope boundary rule

A good boundary rule is:

If a source artifact changes what code, structure, contracts, or proof targets the repo must contain, it is in scope. If it only affects downstream operational life after the generated repo is handed off, it is outside scope unless explicitly required as a build artifact.

That is the clean boundary.

---

14. Projection completeness rule

The Build Projection Pack should be considered in-scope complete only when all build-relevant kit layers have been translated into the target projection classes needed for:

- repo blueprint derivation
- file inventory derivation
- build-plan derivation
- verification targeting

If one of those downstream derivations still requires major guessing, projection scope has not been fully satisfied.

---

15. Why this section matters

Without a clear projection scope:

- the pack will drift back toward brief-only extraction
- architecture may be read but not projected
- security may be present but not materialized
- data/API/design may remain descriptive
- file inventory will stay weak
- the repo will stay underbuilt

BPP-01 makes projection comprehensive enough to be useful.

---

16. Success definition

BPP-01 is successful when it clearly defines that the Build Projection Pack must translate all build-relevant Agent Kit truth—across brief, canonical, work breakdown, acceptance, architecture, implementation, design, data, API, security, and relevant ops/governance layers—into explicit module, interface, data, UI, file, trace, and repo projections that can drive blueprinting and generation without major guessing.

---

17. Short version for the doc

BPP-01 — Projection Scope

The Build Projection Pack covers all Agent Kit truth that materially affects what the repository must contain, how it must be structured, and what it must prove. Its scope includes projecting brief, canonical, work breakdown, acceptance, architecture, implementation, design, data, API, security, and relevant ops/governance layers into explicit module, interface, data, UI, file, trace, and repo structures that the internal build system can execute against.

BPP-02 — Source-to-Projection Mapping

BPP-02 — Source-to-Projection Mapping

1. Purpose

Define how each Agent Kit source layer must be translated into specific Build Projection outputs.

BPP-02 exists so the projection layer is not allowed to vaguely say “we used the kit.” It must show, in a structured way, which source artifacts feed which projection artifacts.

This is the section that turns projection from:

- general intent

into:

- explicit compilation rules

---

2. Core rule

Every build-relevant source layer in the Agent Kit must map to one or more explicit projection targets.

That means the system should never read a source pack only “for context” if that pack changes what the repo should contain. If a source layer affects implementation, structure, contracts, or proof, it must be mapped into a projection artifact.

---

3. Main mapping principle

The main principle of BPP-02 is:

Source artifacts do not go directly to generation. They first map into explicit build-facing projections.

So the system flow becomes:

Agent Kit source
→ projection mapping

→ projection artifacts
→ repo blueprint
→ build plan
→ generation

That mapping layer is what stops knowledge loss.

---

4. Mapping model

The Build Projection Pack should use a source-to-projection mapping model with three levels:

Level 1 — Source layer

What part of the Agent Kit is being consumed.

Level 2 — Projection target

What build-facing projection artifact it feeds.

Level 3 — Derived build truth

What concrete repo/build truth is produced from that projection.

This creates a clear translation chain.

---

5. Source-to-projection mapping table

5.1 Build Brief

Source role

Defines app identity, priorities, and high-level delivery intent.

Must map into

- repo projection
- module projection
- file projection

Derived build truth

- app name
- app type
- stack orientation
- repo type
- repo shape
- high-level subsystem emphasis

The brief should not stay just a narrative intro. It must influence repo identity.

5.2 Canonical Spec

Source role

Defines the normalized, authoritative truth of the app.

Must map into

- module projection
- data projection
- interface projection
- repo projection

Derived build truth

- core domain concepts
- normalized entities
- primary subsystem boundaries

- resolved assumptions
- product truth that shapes implementation

The canonical spec is one of the main build-authority inputs.

5.3 Work Breakdown

Source role

Defines feature decomposition and work-unit structure.

Must map into

- module projection
- UI projection
- interface projection
- trace projection
- file projection

Derived build truth

- feature groups
- work-unit implementation clusters
- subsystem assignment hints
- build slices
- test/proof targeting groups

Work breakdown should help determine what gets built as distinct implementation units.

5.4 Acceptance Map

Source role

Defines what must be true for the system to count as complete.

Must map into

- trace projection
- verification projection
- file projection

Derived build truth

- proof targets
- test obligations
- verification coverage requirements
- acceptance-to-module mapping
- acceptance-to-file/test mapping

Acceptance cannot remain abstract. It must become build-driving proof structure.

5.5 Architecture Pack

Source role

Defines subsystem, layer, and dependency structure.

Must map into

- module projection

- repo projection
- file projection
- interface projection

Derived build truth

- subsystem list
- module boundaries
- dependency direction
- layer layout
- service boundaries
- shared/common package structure

This is one of the strongest drivers of repo structure.

5.6 Implementation Pack

Source role

Defines technical implementation expectations.

Must map into

- module projection
- interface projection
- file projection
- trace projection

Derived build truth

- services
- handlers
- flows
- implementation-specific components
- technical obligations per feature
- file-class expectations

This is where many concrete code targets should come from.

5.7 Design Pack

Source role

Defines UI/UX structure and interaction behavior.

Must map into

- UI projection
- file projection
- trace projection

Derived build truth

- pages/screens
- layouts
- component families
- interaction states
- UI-level flows

- view-level file targets

The design pack should not remain a moodboard. It should become UI build structure.

5.8 Data Pack

Source role

Defines domain data and persistence structure.

Must map into

- data projection
- module projection
- file projection
- interface projection

Derived build truth

- entities
- schemas
- relationships
- migrations
- repositories/data services
- persistence-layer modules
- validation/data contract files

This is one of the main sources of backend depth.

5.9 API Pack

Source role

Defines external and internal request/response surfaces.

Must map into

- interface projection
- module projection
- file projection
- trace projection

Derived build truth

- endpoint inventory
- route handlers
- request/response types
- integration contracts
- event/webhook surfaces
- API-layer test targets

This should strongly shape backend and shared contracts.

5.10 Security Pack

Source role

Defines auth, authorization, and protection requirements.

Must map into

- module projection
- interface projection
- file projection
- trace projection

Derived build truth

- auth modules
- RBAC/permission structures
- middleware/security guards
- sensitive data handling rules
- protected endpoints
- security-related tests and verification targets

Security should not stay “policy-only.” It must become code-facing structure.

5.11 Ops / Governance Pack

Source role

Defines build-relevant operational and governance obligations.

Must map into

- file projection
- verification projection
- trace projection
- repo projection where relevant

Derived build truth

- audit-related artifacts
- config expectations
- logging/tracing obligations if required in build scope
- operational scripts
- proof/record outputs
- governed supporting files

Not all ops/governance content is build-relevant, but the part that is should project forward.

---

6. Projection target definitions

6.1 Module Projection

Collects everything that becomes:

- subsystems
- services
- modules
- handlers
- jobs
- adapters
- middleware families

Primary source feeds:

- canonical

- work breakdown
- architecture
- implementation
- data
- API
- security

6.2 UI Projection

Collects everything that becomes:

- screens
- pages
- layouts
- component families
- interaction-state surfaces

Primary source feeds:

- work breakdown
- design
- implementation
- acceptance

6.3 Interface Projection

Collects everything that becomes:

- routes
- endpoints
- events
- contracts
- request/response surfaces

Primary source feeds:

- canonical
- implementation
- API
- data
- security

6.4 Data Projection

Collects everything that becomes:

- entities
- schemas
- storage types
- migrations
- relationships

Primary source feeds:

- canonical
- data
- implementation
- API

6.5 File Projection

Collects everything that becomes:

- directories
- file classes
- concrete file targets
- file volume expectations

Primary source feeds:

- almost every source layer, especially architecture, implementation, design, data, API, security, and acceptance

---

6.6 Trace Projection

Collects everything that becomes:

- requirement-to-module map
- requirement-to-file map
- acceptance-to-proof map
- acceptance-to-test map

Primary source feeds:

- work breakdown
- acceptance
- implementation
- API
- security
- design

6.7 Repo Projection

Collects everything that becomes:

- repo type
- repo shape
- layer placement
- package structure
- subsystem placement

Primary source feeds:

- brief
- canonical
- architecture
- implementation
- ops/governance where structurally relevant

---

7. Required mapping behavior

The Build Projection Pack should enforce these mapping behaviors:

7.1 No unused build-relevant source layer

If a source layer materially affects implementation, it must map to at least one projection target.

7.2 No projection target without source basis

Every projection target should be attributable to one or more source layers.

7.3 No silent projection gaps

If a source layer exists but cannot be mapped, the system should flag the gap or fail projection.

7.4 No over-centralization

The brief alone must not dominate projection. Projection must pull from the deeper kit layers.

These rules prevent the current shallow-build behavior.

---

8. Minimum mapping outputs

At minimum, the source-to-projection mapping step should produce a structured record that shows:

- source layer name
- source artifact refs
- projection targets fed
- derived objects created
- unresolved projection gaps if any

This can later become its own artifact like:
`source_projection_map.json`

---

9. Example mapping fragment

A simple example could look like:

```
{
 "source_layer": "security_pack",
 "source_refs": [
   "SEC-01-auth-model.md",
   "SEC-02-rbac-rules.md",
   "SEC-03-data-protection.md"
 ],
 "projection_targets": [
   "module_projection",
   "interface_projection",
   "file_projection",
   "trace_projection"
 ],
 "derived_outputs": {
   "modules": [
     "auth_service",
     "permission_guard_middleware"
   ],
   "interfaces": [
     "protected_document_routes"
   ],
   "files": [
     "src/server/middleware/authGuard.ts",
     "src/server/middleware/permissionGuard.ts",
     "src/server/services/authService.ts"
   ],
   "trace_entries": 14
 },
 "projection_gaps": []
}
```

That is the kind of record you want.

---

10. Projection gap rule

BPP-02 should require that any source layer that fails to map cleanly produces a projection-gap record.

Examples:

- architecture pack defines subsystem but no module projection could be derived
- design pack defines workflow but no UI projection target was created
- acceptance rule exists but no proof target could be mapped

Those gaps should be visible, not silently swallowed.

---

11. Why this section matters

Without explicit source-to-projection mapping:

- the kit gets “used” in name only
- the brief dominates
- deep packs become advisory instead of generative
- architecture/security/data/API truth fails to materialize
- file inventory stays too small
- repo structure stays generic

BPP-02 is what forces the system to actually honor the whole Agent Kit.

---

12. Success definition

BPP-02 is successful when every build-relevant Agent Kit source layer is explicitly mapped into one or more projection targets, each projection target is attributable to actual source artifacts, and the system can show how descriptive kit truth became build-facing module, UI, interface, data, file, trace, and repo structures without major silent loss.

---

13. Short version for the doc

BPP-02 — Source-to-Projection Mapping
The Build Projection Pack must explicitly map each build-relevant Agent Kit source layer into

one or more build-facing projection targets. Brief, canonical, work breakdown, acceptance, architecture, implementation, design, data, API, security, and relevant ops/governance artifacts must be translated into module, UI, interface, data, file, trace, and repo projections so the internal build system can prove how kit truth became executable repo structure.

BPP-03 — Projection Artifacts

BPP-03 — Projection Artifacts

1. Purpose

Define the concrete artifact set that makes up the Build Projection Pack.

BPP-03 exists to turn the projection layer into real objects rather than a vague process. If BPP-02 defines how source layers map into projections, BPP-03 defines the actual projection artifacts the system must produce and persist.

These artifacts are what the Repo Blueprint and Build Plan should consume.

---

2. Core rule

The Build Projection Pack must produce explicit, inspectable artifacts.

It is not enough for the planner to “internally understand” the kit. Projection outputs must exist as real structured artifacts that can be:

- validated
- inspected
- audited
- reused by downstream build steps
- compared against generated output

---

3. Main artifact principle

The main principle of BPP-03 is:

Every major build-facing dimension of the Agent Kit must have a corresponding projection artifact.

That means the projection layer should produce artifacts for:

- modules
- UI
- interfaces
- data
- files
- traceability
- repo structure seed

That is the minimum complete set.

---

4. Required projection artifacts

At minimum, the Build Projection Pack should contain these artifacts:

1. `module_inventory`
2. `ui_inventory`
3. `endpoint_inventory`
4. `entity_inventory`
5. `file_inventory`
6. `requirement_trace_map`
7. `verification_trace_map`
8. `repo_blueprint_seed`
9. `source_projection_map`

10. `kit_extraction_report`

That is the core Build Projection Pack.

---

5. Artifact 1 — `module_inventory`

Purpose

Defines the subsystem and module structure the repo must contain.

What it should capture

- subsystem list
- module list
- module purpose
- module layer
- module dependencies
- ownership/responsibility boundaries

Why it matters

This is one of the main anti-drift artifacts because it stops the build from flattening everything into feature prose.

Typical contents

- auth modules
- document modules
- collaboration modules
- AI orchestration modules
- search modules

- settings/admin modules
- audit/logging modules where in scope

---

6. Artifact 2 — `ui_inventory`

Purpose

Defines the UI-facing structure that must be materialized.

What it should capture

- screens/pages
- layouts
- component families
- major interaction states
- screen-to-feature relationships

Why it matters

Without this artifact, design tends to collapse into generic pages and broad components.

Typical contents

- login page
- dashboard page
- editor page
- settings page
- workspace switcher
- document sidebar

- collaboration indicators
- AI suggestion surfaces

---

7. Artifact 3 — `endpoint_inventory`

Purpose

Defines the route and interface surfaces the backend/shared layers must support.

What it should capture

- endpoints
- route handlers
- request/response contracts
- events/webhooks if relevant
- protected/public surface classification

Why it matters

This prevents the backend from being generated as vague models/routes with weak interface discipline.

Typical contents

- route path
- method
- purpose
- auth requirement
- request schema ref
- response schema ref

- owning module

---

8. Artifact 4 — `entity_inventory`

Purpose

Defines the domain data model that must be materialized into schemas, models, contracts, and migrations.

What it should capture

- entities
- fields
- relationships
- persistence model
- migration implications
- validation surfaces

Why it matters

This is one of the main sources of backend depth and shared-contract seriousness.

Typical contents

- User
- Workspace
- Document
- Comment
- Suggestion
- Version

- Notification
- PermissionAssignment

---

9. Artifact 5 — `file_inventory`

Purpose

Defines the expected concrete file targets or file-target classes for the repo.

What it should capture

- expected directories
- expected file classes
- expected concrete files
- file ownership by module/layer
- expected file counts by area
- total expected file count

Why it matters

This is the artifact that directly addresses the underbuilt-repo problem.

Without it, the generator stops when it feels done.
With it, the generator stops when the inventory is complete.

Typical contents

- `src/server/routes/document.ts`
- `src/server/services/documentService.ts`
- `src/app/pages/Editor.tsx`

- `src/app/components/editor/EditorToolbar.tsx`
- `src/shared/contracts/document.ts`
- `tests/integration/documentRoutes.test.ts`

---

10. Artifact 6 — `requirement_trace_map`

Purpose

Maps requirements and work items to modules/files.

What it should capture

- requirement ID
- source artifact ref
- mapped module(s)
- mapped file(s)
- mapped endpoint(s) or UI surface(s)

Why it matters

This proves the build is actually using the Agent Kit rather than only summarizing it.

Typical shape

`requirement → module → file`

---

11. Artifact 7 — `verification_trace_map`

Purpose

Maps acceptance and proof obligations to tests, checks, and files.

What it should capture

- acceptance rule ID
- proof obligation
- mapped test targets
- mapped verification categories
- mapped file/module surfaces

Why it matters

This makes acceptance actionable and prevents verification from staying abstract.

Typical shape

`acceptance criterion → proof target → test/check/file`

---

12. Artifact 8 — `repo_blueprint_seed`

Purpose

Provides the projection-layer seed for the full Repo Blueprint.

What it should capture

- repo type
- repo shape
- subsystem placement
- layer layout
- package/folder placement hints
- structural implications from inventories

Why it matters

This is the immediate bridge from projection artifacts to the Repo Blueprint.

It should not be the full Repo Blueprint yet, but it should carry enough structural truth to derive one.

---

13. Artifact 9 — `source_projection_map`

Purpose

Shows how Agent Kit source layers were translated into projection artifacts.

What it should capture

- source layer
- source refs
- projection targets fed
- derived outputs
- projection gaps if any

Why it matters

This is the proof that the whole kit was actually used.

Without it, you cannot really verify deep extraction happened.

---

14. Artifact 10 — `kit_extraction_report`

Purpose

Summarizes what was extracted from the Agent Kit and whether extraction is complete enough to proceed.

What it should capture

- kit ID/ref
- layers extracted

- source files consumed
- extraction counts
- unresolved extraction gaps
- expected file count derived
- extraction pass/fail status

Why it matters

This is the gate artifact that prevents repo generation from starting on shallow kit consumption.

---

15. Artifact relationships

These projection artifacts should work together, not stand alone.

A clean relationship model is:

- `kit_extraction_report` proves extraction happened
- `source_projection_map` proves source-to-projection translation happened
- `module_inventory`, `ui_inventory`, `endpoint_inventory`, `entity_inventory` define the executable build surface
- `file_inventory` defines the materialization target
- `requirement_trace_map` and `verification_trace_map` define traceability and proof
- `repo_blueprint_seed` feeds the full Repo Blueprint

That is the full Build Projection Pack circuit.

---

16. Minimum required fields per artifact

At a minimum, each projection artifact should have:

- artifact ID
- run ID
- kit ref
- created_at
- updated_at
- source_refs
- status

And then its domain-specific fields.

This keeps them all traceable back to the same build authority chain.

---

17. Required persistence rule

These artifacts should not remain ephemeral planner memory.

They should be written as runtime artifacts under the run.

A clean path would be:

```
Axion/.axion/runs/<RUN_ID>/build/projection/
 kit_extraction_report.json
 source_projection_map.json
 module_inventory.json
 ui_inventory.json
 endpoint_inventory.json
 entity_inventory.json
 file_inventory.json
 requirement_trace_map.json
 verification_trace_map.json
 repo_blueprint_seed.json
```

That makes them inspectable and reusable.

---

18. Artifact validation rule

Each projection artifact should be schema-validatable and structurally strict.

That means later you should create schemas for them, just like Build Mode artifacts, so the projection layer cannot become loose or hand-wavy.

---

19. What this section prevents

BPP-03 prevents:

- brief-only builds
- hidden planner assumptions
- architecture loss during translation
- design/API/data/security packs being ignored
- file-inventory underbuilding
- weak acceptance traceability

It turns projection into a real subsystem instead of a soft concept.

---

20. Success definition

BPP-03 is successful when the Build Projection Pack is defined as a concrete artifact set containing module, UI, endpoint, entity, file, trace, repo-seed, extraction, and source-mapping artifacts that can be persisted, validated, inspected, and used to drive Repo Blueprint derivation and downstream repo generation.

---

21. Short version for the doc

BPP-03 — Projection Artifacts
The Build Projection Pack consists of a concrete artifact set: `module_inventory`, `ui_inventory`, `endpoint_inventory`, `entity_inventory`, `file_inventory`, `requirement_trace_map`, `verification_trace_map`, `repo_blueprint_seed`,

`source_projection_map`, and `kit_extraction_report`. Together, these artifacts convert Agent Kit truth into explicit build-facing structures that can be validated, persisted, inspected, and used to derive the Repo Blueprint and control repo generation.

BPP-04 — Module Inventory

BPP-04 — Module Inventory

1. Purpose

Define the `module_inventory` artifact as the projection artifact that turns Agent Kit truth into explicit subsystem and module structure.

BPP-04 exists because one of the biggest reasons builds collapse is that the system understands features, but not the actual module architecture the repo must contain. The `module_inventory` fixes that by making module structure explicit before the Repo Blueprint and Build Plan are derived.

---

2. Core role

`module_inventory` is the artifact that defines:

- what subsystems exist
- what modules exist inside those subsystems
- what each module is responsible for
- what layer each module belongs to
- how modules depend on one another

It is the main build-facing decomposition of the application into implementation units.

---

3. Main goal

The main goal of `module_inventory` is to prevent the builder from reducing rich kit knowledge into broad feature blobs.

Instead of:

- “documents feature”

- “collaboration feature”
- “AI feature”

the system should know:

- document service
- document repository
- document routes
- document editor page
- document validation layer
- collaboration session service
- collaboration event transport
- suggestion engine adapter
- suggestion review UI
- permission guard middleware

That is the level of decomposition this artifact should force.

---

4. Why this artifact is needed

A build system that does not have an explicit module inventory will usually do one of two bad things:

4.1 Over-compress

Collapse many requirements into a few oversized files or vague feature folders.

4.2 Over-infer

Invent module structure during generation based on weak heuristics.

`module_inventory` prevents both by making the module graph explicit before generation begins.

---

5. What this artifact represents

`module_inventory` represents the **implementation architecture of the app in module form**.

It is not:

- the full repo blueprint
- the full file inventory
- the acceptance map
- the endpoint inventory
- the entity inventory

It is the module-layer artifact that sits between architecture truth and file truth.

---

6. What this artifact consumes

The `module_inventory` should be derived from these Agent Kit layers:

- canonical spec
- work breakdown
- architecture pack
- implementation pack
- data pack
- API pack
- security pack

- design pack where UI modules are involved

It may be influenced by the brief, but the brief alone should never define the module inventory.

---

7. What this artifact produces

The `module_inventory` should produce explicit definitions for:

- subsystems
- modules
- module layer
- module purpose
- module responsibilities
- module dependencies
- module interfaces
- module ownership boundaries
- module-to-requirement mapping
- module-to-file-class implications

This makes it one of the strongest drivers of repo depth.

---

8. Required artifact structure

At minimum, `module_inventory` should include these top-level sections:

1. artifact identity
2. subsystem list

3. module list
4. dependency map
5. requirement links
6. generation implications

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `module_inventory_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it traceable and aligned with the rest of the Build Projection Pack.

---

10. Subsystem model

The artifact should define explicit subsystems.

Each subsystem should include:

- `subsystem_id`
- `name`
- `purpose`
- `layer_scope`
- `module_ids`

Examples of subsystems:

- `frontend_app`
- `backend_api`
- `shared_contracts`
- `realtime_collaboration`
- `auth_and_access`
- `search_and_discovery`
- `ai_assistance`
- `ops_and_audit`

The exact list depends on the kit, but the artifact should force subsystem boundaries to be visible.

---

11. Module model

Each module entry should be explicit and structured.

At minimum, each module should include:

- `module_id`
- `name`
- `subsystem_id`
- `layer`
- `purpose`
- `responsibilities`
- `depends_on`
- `interfaces`
- `source_requirement_refs`
- `source_artifact_refs`
- `file_class_targets`
- `status`

This is the core of the artifact.

---

12. Module layer classification

Each module should have a declared layer.

Suggested layer values:

- `frontend`
- `backend`
- `shared`

- `data`
- `security`
- `integration`
- `realtime`
- `ops`
- `test_support`

This helps later with:

- repo placement
- blueprint derivation
- file inventory generation

---

13. Module purpose rule

Every module must have a clear purpose statement.

Bad example:

- “handles docs”

Good example:

- “Owns document lifecycle operations including create, load, update, archive, and version transitions.”

This matters because vague module purposes cause vague file generation later.

---

14. Module responsibility rule

Every module should list concrete responsibilities.

For example, `document_service` might own:

- document create
- document update
- document load
- document archive
- document version fetch

This prevents module definitions from being only labels.

---

15. Dependency rule

Every module should explicitly list its dependencies.

This should include:

- internal module dependencies
- shared contract dependencies
- data dependencies
- service dependencies where relevant

This helps the blueprint derive:

- import direction
- layer ordering
- generation order

It also prevents accidental circular or guessed dependencies later.

---

16. Interface rule

Every module should declare its interface surfaces where applicable.

Examples:

- API routes
- service methods
- events
- shared contracts
- UI surface responsibilities
- middleware hooks

This helps connect `module_inventory` to:

- endpoint inventory
- UI inventory
- entity inventory
- trace maps

---

17. Source linkage rule

Every module should be attributable back to the Agent Kit.

At minimum, each module should carry:

- `source_requirement_refs`
- `source_artifact_refs`

That means no major module should exist without a source basis in the kit.

This is one of the most important anti-drift rules in the entire Build Projection Pack.

---

18. File-class target rule

Each module should include the kinds of files it implies.

Example for a backend service module:

- service file
- route file
- validation file
- contract file
- test file

Example for a UI module:

- page file
- component file
- state/hook file
- test file

This is how `module_inventory` feeds `file_inventory`.

---

19. Dependency map section

Beyond per-module dependencies, the artifact should also provide a dependency map summary.

This can include:

- subsystem dependency edges

- module dependency edges
- restricted dependency directions

Examples:

- frontend may depend on shared, not backend internals
- routes may depend on services, not database adapters directly
- UI components may depend on shared types and hooks, not server modules

This helps preserve architecture during generation.

---

20. Requirement links section

The artifact should summarize how requirements map into modules.

At minimum, it should support:

- requirement ID
- mapped module IDs

This is not the full requirement trace map, but it gives the module inventory a direct connection to requirement coverage.

---

21. Generation implications section

The artifact should also summarize what generation should expect from the module inventory.

Examples:

- total subsystem count
- total module count
- module count by layer

- expected module-driven file classes
- expected module-driven file count contribution

This is important because it helps quantify how rich the module layer really is.

---

22. Example module entry

A simple example:

```
{
 "module_id": "document_service",
 "name": "Document Service",
 "subsystem_id": "backend_api",
 "layer": "backend",
 "purpose": "Owns document lifecycle operations and application-level document business
logic.",
 "responsibilities": [
   "create_document",
   "load_document",
   "update_document",
   "archive_document",
   "fetch_document_versions"
 ],
 "depends_on": [
   "document_repository",
   "workspace_permission_guard",
   "document_contracts"
 ],
 "interfaces": [
   "service_api",
   "document_routes"
 ],
 "source_requirement_refs": [
   "REQ-DOC-001",
   "REQ-DOC-002"
 ],
 "source_artifact_refs": [
   "IMPL-document-lifecycle.md",
   "ARCH-backend-service-boundaries.md"
 ],
 "file_class_targets": [
```

```
  "service",
  "route",
  "validation",
  "shared_contract",
  "unit_test",
  "integration_test"
 ],
 "status": "active"
}
```

---

23. Example subsystem entry

```
{
 "subsystem_id": "backend_api",
 "name": "Backend API",
 "purpose": "Owns the server-side application layer, API routes, services, and server-side policy enforcement.",
 "layer_scope": ["backend", "security", "data"],
 "module_ids": [
  "document_service",
  "workspace_service",
  "auth_service",
  "permission_guard_middleware"
 ]
}
```

---

24. Anti-drift rules

`module_inventory` should enforce these rules:

24.1 No source-less major modules

A major module cannot appear without source linkage.

24.2 No feature-only decomposition

Feature names alone are not enough; modules must represent implementation units.

24.3 No vague modules

A module must have a concrete purpose and responsibilities.

24.4 No missing dependencies

Modules should not be left dependency-blank if they clearly depend on other system parts.

24.5 No architecture-loss compression

If the architecture pack implies multiple distinct modules, the inventory must not collapse them into one generic bucket.

These rules stop the exact under-extraction problem you found.

---

25. Relationship to other projection artifacts

`module_inventory` should connect directly to:

- `entity_inventory`
- `endpoint_inventory`
- `ui_inventory`
- `file_inventory`
- `requirement_trace_map`
- `repo_blueprint_seed`

It should be one of the main drivers of the Repo Blueprint.

---

26. Completion rule

`module_inventory` should only be considered complete when:

- all major subsystems implied by the kit are represented
- all major implementation modules implied by the kit are represented
- each module has purpose, responsibilities, dependencies, and source linkage
- the artifact is rich enough to drive file inventory derivation

If it cannot do that, it is incomplete.

---

27. Success definition

BPP-04 is successful when `module_inventory` is defined as a structured projection artifact that decomposes Agent Kit truth into explicit subsystems and implementation modules with clear purposes, responsibilities, dependencies, source linkage, and file-class implications, so the internal build system can derive real repo structure instead of compressing the app into shallow feature scaffolding.

---

28. Short version for the doc

BPP-04 — Module Inventory
`module_inventory` is the Build Projection artifact that defines the subsystem and module architecture the repo must contain. It maps Agent Kit truth into explicit implementation units with declared layers, responsibilities, dependencies, source references, and file-class implications. Its purpose is to prevent architectural collapse during generation and to give the Repo Blueprint and File Inventory a concrete module structure to build from.

BPP-05 — UI Inventory

BPP-05 — UI Inventory

1. Purpose

Define the `ui_inventory` artifact as the projection artifact that turns design, workflow, and feature truth from the Agent Kit into explicit UI build structure.

BPP-05 exists because many build systems understand that an app has “screens” or “features,” but they do not turn that into a disciplined UI surface. The result is generic pages, weak component breakdown, and missing interaction states.

`ui_inventory` fixes that by making the UI build surface explicit before the Repo Blueprint and File Inventory are derived.

---

2. Core role

`ui_inventory` is the artifact that defines:

- what screens/pages exist
- what layouts exist
- what major component families exist
- what interaction states matter
- what UI flows connect those surfaces
- what UI responsibilities belong to which screens/components

It is the main build-facing decomposition of the frontend and user-facing experience into implementation units.

---

3. Main goal

The main goal of `ui_inventory` is to stop the build from reducing rich UX/product/design truth into a handful of generic pages.

Instead of:

- dashboard page
- editor page
- settings page

the system should know:

- workspace dashboard shell
- document list panel
- editor workspace
- collaborator presence rail
- AI suggestion panel
- version history drawer
- comment thread panel
- document settings modal
- onboarding flow surfaces

That is the level of decomposition this artifact should force.

---

4. Why this artifact is needed

A build system without an explicit UI inventory usually does one of two bad things:

4.1 Over-simplify

Turn many workflows into a few oversized pages with weak structure.

4.2 Over-improvise

Invent component/page structure during generation based on naming heuristics.

`ui_inventory` prevents both by making the UI surface explicit before generation starts.

---

5. What this artifact represents

`ui_inventory` represents the **user-facing implementation surface of the app**.

It is not:

- the design pack itself
- the full file inventory
- the endpoint inventory
- the module inventory
- the acceptance map

It is the UI-layer projection artifact that sits between design/workflow truth and file truth.

---

6. What this artifact consumes

The `ui_inventory` should be derived from these Agent Kit layers:

- work breakdown
- design pack
- implementation pack
- canonical spec
- acceptance map
- security pack where UI permissions or protected flows matter
- API pack where UI-surface contracts matter

The brief may influence it, but the UI inventory should not be brief-driven alone.

---

7. What this artifact produces

The `ui_inventory` should produce explicit definitions for:

- pages/screens
- layouts/shells
- major components/component families
- UI states
- protected/public UI surfaces
- UI flow relationships
- screen-to-feature mappings
- screen/component-to-file-class implications

This makes it one of the strongest drivers of frontend depth.

---

8. Required artifact structure

At minimum, `ui_inventory` should include these top-level sections:

1. artifact identity
2. page/screen list
3. layout list
4. component-family list
5. interaction-state list
6. UI flow links
7. requirement and acceptance links

8. generation implications

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `ui_inventory_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Page / screen model

Each page or screen entry should be explicit.

At minimum, each page should include:

- `screen_id`
- `name`
- `surface_type`

- `purpose`
- `route_ref` or `navigation_ref`
- `layout_id`
- `component_family_ids`
- `ui_states`
- `source_requirement_refs`
- `source_artifact_refs`
- `file_class_targets`
- `status`

Suggested `surface_type` values:

- `page`
- `panel`
- `modal`
- `drawer`
- `wizard`
- `embedded_surface`

This stops all UI surfaces from being flattened into “pages.”

---

11. Layout model

Each layout or shell should be explicit.

At minimum:

- `layout_id`
- `name`
- `purpose`
- `applies_to_screen_ids`
- `regions`

Examples of `regions`:

- header
- sidebar
- content
- footer
- secondary_panel
- activity_rail

This matters because many serious apps are layout-rich, not just page-rich.

---

12. Component-family model

The artifact should define component families, not only individual leaf widgets.

At minimum, each component family should include:

- `component_family_id`

- `name`
- `purpose`
- `screen_ids`
- `interaction_role`
- `source_requirement_refs`
- `file_class_targets`

Examples:

- editor_toolbar
- document_sidebar
- collaborator_presence
- ai_suggestion_surface
- workspace_switcher
- permission_guard_ui
- onboarding_step_components

This helps derive coherent component directories and file targets.

---

13. UI state model

Each significant UI surface should declare meaningful interaction states.

Examples:

- empty

- loading
- success
- error
- locked
- read_only
- collaborator_active
- ai_processing
- validation_failed
- permission_denied

This matters because real UI is not just static page structure.

These states should later feed:

- component props/state modeling
- tests
- acceptance mapping

---

14. UI flow rule

The artifact should identify important UI flow relationships.

Examples:

- login → workspace selection → dashboard
- dashboard → document editor
- editor → version history drawer

- editor → comment panel
- settings → billing or member management
- protected route redirects

This does not need to become a full flow engine, but it should preserve major navigational and interaction relationships.

---

15. Source linkage rule

Every major UI surface should be attributable back to the Agent Kit.

At minimum:

- `source_requirement_refs`
- `source_artifact_refs`

That means no major page/component family should appear without source basis.

This is one of the main anti-drift rules for frontend generation.

---

16. File-class target rule

Each UI artifact should include what kinds of files it implies.

Examples for a page:

- page file
- state/hook file
- route registration file
- test file

Examples for a component family:

- component file(s)
- style/composition file if applicable
- story/demo file if in scope
- unit test file

This is how `ui_inventory` feeds `file_inventory`.

---

17. Protected UI rule

The inventory should explicitly identify protected or role-sensitive surfaces.

Examples:

- admin settings pages
- member management
- billing surfaces
- workspace-only content
- editor actions restricted by role

This helps the build system connect UI projection to:

- security projection
- route guards
- permission-aware rendering
- acceptance checks

---

18. Requirement and acceptance links

The artifact should summarize how requirements and acceptance obligations map into UI surfaces.

At minimum, it should support:

- requirement ID → screen/component family
- acceptance criterion → UI state/flow target

This is not the full trace map, but it keeps the UI inventory grounded in actual kit obligations.

---

19. Generation implications section

The artifact should summarize what generation should expect from the UI layer.

Examples:

- total screen count
- total layout count
- total component family count
- total UI state count
- expected UI-driven file classes
- expected UI-driven file count contribution

This matters because it helps quantify whether the frontend is being underbuilt.

---

20. Example page entry


```
{
 "screen_id": "editor_workspace",
 "name": "Editor Workspace",
 "surface_type": "page",
 "purpose": "Primary document editing surface with collaboration, comments, and AI-assisted refinement.",
 "route_ref": "/documents/:id",
```

"layout_id": "workspace_shell",
"component_family_ids": [
  "editor_toolbar",
  "document_canvas",
  "collaborator_presence",
  "ai_suggestion_surface",
  "comment_thread_panel"
],
"ui_states": [
  "loading",
  "ready",
  "read_only",
  "collaborator_active",
  "ai_processing",
  "error"
],
"source_requirement_refs": [
  "REQ-EDIT-001",
  "REQ-COLLAB-002",
  "REQ-AI-003"
],
"source_artifact_refs": [
  "DES-editor-experience.md",
  "IMPL-editor-workflow.md"
],
"file_class_targets": [
  "page",
  "hook",
  "component_family",
  "route_registration",
  "integration_test"
],
"status": "active"
}

---

21. Example layout entry

{
"layout_id": "workspace_shell",
"name": "Workspace Shell",
"purpose": "Shared authenticated workspace layout with navigation, content area, and secondary collaboration panels.",
"applies_to_screen_ids": [
  "workspace_dashboard",

```
  "editor_workspace",
  "workspace_settings"
 ],
 "regions": [
  "header",
  "sidebar",
  "content",
  "secondary_panel"
 ]
}
```

---

22. Example component family entry

```
{
 "component_family_id": "ai_suggestion_surface",
 "name": "AI Suggestion Surface",
 "purpose": "Displays and applies AI-generated document suggestions with review controls and status feedback.",
 "screen_ids": ["editor_workspace"],
 "interaction_role": "review_and_apply_assistance",
 "source_requirement_refs": [
  "REQ-AI-003",
  "REQ-AI-004"
 ],
 "file_class_targets": [
  "component",
  "hook",
  "state_model",
  "unit_test"
 ]
}
```

---

23. Anti-drift rules

`ui_inventory` should enforce these rules:

23.1 No source-less major UI surfaces

A major page, layout, or component family cannot exist without source linkage.

23.2 No page-only compression

Complex workflows must not be collapsed into one vague page if the kit implies distinct surfaces.

23.3 No design-loss flattening

If the design pack implies layouts, panels, drawers, or modals, the inventory must preserve them.

23.4 No missing UI-state modeling

Critical states must not be omitted where they affect acceptance or interaction logic.

23.5 No security-blind surfaces

Protected or role-sensitive UI areas must be flagged as such.

These rules stop the frontend from becoming shallow and generic.

---

24. Relationship to other projection artifacts

`ui_inventory` should connect directly to:

- `module_inventory`
- `endpoint_inventory`
- `requirement_trace_map`
- `verification_trace_map`
- `file_inventory`
- `repo_blueprint_seed`

It should be one of the main drivers of frontend-related repo structure.

---

25. Completion rule

`ui_inventory` should only be considered complete when:

- all major user-facing surfaces implied by the kit are represented
- each UI surface has purpose, layout/component structure, states, and source linkage
- the artifact is rich enough to drive frontend file inventory derivation
- protected and acceptance-relevant UI flows are visible

If it cannot do that, it is incomplete.

---

26. Success definition

BPP-05 is successful when `ui_inventory` is defined as a structured projection artifact that decomposes Agent Kit truth into explicit pages, layouts, component families, UI states, and flow-linked surfaces with source linkage and file-class implications, so the internal build system can derive a serious frontend structure instead of collapsing the app into a few generic pages.

---

27. Short version for the doc

BPP-05 — UI Inventory
`ui_inventory` is the Build Projection artifact that defines the user-facing structure the repo must contain. It maps Agent Kit design, workflow, implementation, acceptance, and security truth into explicit pages, layouts, component families, UI states, and flow-linked surfaces with source references and file-class implications. Its purpose is to preserve UI depth and interaction structure before repo blueprinting and generation.

BPP-06 — Endpoint Inventory

BPP-06 — Endpoint Inventory

1. Purpose

Define the `endpoint_inventory` artifact as the projection artifact that turns API, workflow, module, and security truth from the Agent Kit into explicit backend and interface surfaces.

BPP-06 exists because many build systems know an app “has an API,” but they do not convert that into a disciplined interface inventory. The result is shallow routes, generic handlers, weak contracts, and backend structure that does not match the actual product.

`endpoint_inventory` fixes that by making the interface surface explicit before the Repo Blueprint and File Inventory are derived.

---

2. Core role

`endpoint_inventory` is the artifact that defines:

- what endpoints exist
- what route handlers exist
- what methods and paths exist
- what contracts those interfaces use
- what auth/permission rules apply
- what modules own each interface
- what events/webhooks exist where relevant

It is the main build-facing decomposition of backend/API surface into implementation units.

---

3. Main goal

The main goal of `endpoint_inventory` is to stop the build from reducing backend/API truth into a handful of vague CRUD routes.

Instead of:

- `/documents`
- `/users`
- `/workspaces`

the system should know:

- `POST /documents`
- `GET /documents/:id`
- `PUT /documents/:id`
- `GET /documents/:id/versions`
- `POST /documents/:id/comments`
- `POST /documents/:id/suggestions`
- `POST /workspaces/:id/invitations`
- `GET /workspaces/:id/members`
- collaboration/session event surfaces
- notification/webhook surfaces if in scope

That is the level of decomposition this artifact should force.

---

4. Why this artifact is needed

A build system without an explicit endpoint inventory usually does one of two bad things:

4.1 Over-simplify

Generate generic routes that do not reflect the real application workflows.

4.2 Over-improvise

Invent backend endpoints during generation based on guessed resource names.

`endpoint_inventory` prevents both by making the interface surface explicit before generation starts.

---

5. What this artifact represents

`endpoint_inventory` represents the **API and event surface the repository must implement**.

It is not:

- the raw API pack
- the full module inventory
- the entity inventory
- the full file inventory
- the verification trace map

It is the interface-layer projection artifact that sits between API/workflow truth and file truth.

---

6. What this artifact consumes

The `endpoint_inventory` should be derived from these Agent Kit layers:

- API pack
- canonical spec
- work breakdown
- implementation pack

- architecture pack
- security pack
- data pack
- acceptance map where interface-level proof matters
- design pack where UI-triggered interface flows matter

The brief may influence it, but the endpoint inventory should never be brief-driven alone.

---

7. What this artifact produces

The `endpoint_inventory` should produce explicit definitions for:

- endpoints
- methods
- paths
- route owners
- request/response contract refs
- auth/permission requirements
- interface purpose
- event/webhook surfaces where relevant
- endpoint-to-module mapping
- endpoint-to-file-class implications

This makes it one of the strongest drivers of backend and shared-contract depth.

---

8. Required artifact structure

At minimum, `endpoint_inventory` should include these top-level sections:

1. artifact identity
2. HTTP/API endpoint list
3. event/webhook surface list
4. route ownership map
5. auth/permission summary
6. requirement and acceptance links
7. generation implications

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `endpoint_inventory_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Endpoint model

Each endpoint entry should be explicit.

At minimum, each endpoint should include:

- `endpoint_id`
- `name`
- `method`
- `path`
- `purpose`
- `owner_module_id`
- `request_contract_ref`
- `response_contract_ref`
- `auth_required`
- `permission_rules`
- `source_requirement_refs`
- `source_artifact_refs`
- `file_class_targets`
- `status`

Suggested `method` values:

- `GET`

- `POST`
- `PUT`
- `PATCH`
- `DELETE`

This stops backend interfaces from being treated as unnamed route guesses.

---

11. Event / webhook model

If the app includes non-HTTP interface surfaces, they should be represented too.

At minimum, each event/webhook entry should include:

- `interface_event_id`
- `name`
- `surface_type`
- `purpose`
- `owner_module_id`
- `trigger`
- `payload_contract_ref`
- `auth_or_trust_boundary`
- `source_requirement_refs`
- `source_artifact_refs`
- `file_class_targets`

- `status`

Suggested `surface_type` values:

- `internal_event`
- `webhook`
- `realtime_event`
- `queue_message`

This matters because serious apps often have more than plain REST routes.

---

12. Route ownership rule

Every endpoint should have a clear owning module.

Examples:

- `document_routes`
- `workspace_service`
- `auth_service`
- `notification_webhook_handler`
- `collaboration_session_gateway`

This connects `endpoint_inventory` directly to `module_inventory` and prevents the API surface from floating separately from system structure.

---

13. Purpose rule

Every endpoint or event surface must have a clear purpose statement.

Bad example:

- “document endpoint”

Good example:

- “Creates a new document in the active workspace and returns the initialized document summary.”

This matters because vague purposes lead to vague route generation and weak tests.

---

14. Contract rule

Each interface surface should reference or imply explicit contracts.

For example:

- request body schema
- path param schema
- query schema
- response type
- event payload type

This helps connect `endpoint_inventory` to:

- shared contracts
- entity inventory
- file inventory
- verification

Without contracts, routes become shallow scaffolds.

---

15. Auth and permission rule

Each endpoint should declare whether:

- auth is required
- what permission scope applies
- whether role-based restrictions exist
- whether it is public, authenticated, workspace-scoped, admin-scoped, or system-scoped

This keeps backend generation aligned with security truth and prevents a generic open API surface.

---

16. Source linkage rule

Every major interface surface should be attributable back to the Agent Kit.

At minimum:

- `source_requirement_refs`
- `source_artifact_refs`

That means no major endpoint/event should appear without source basis.

This is one of the main anti-drift rules for backend/API generation.

---

17. File-class target rule

Each interface surface should include what kinds of files it implies.

Examples for an HTTP endpoint:

- route file

- handler/service file
- request contract file
- response contract file
- integration test file

Examples for an event/webhook:

- event contract file
- handler file
- adapter/integration file
- test file

This is how `endpoint_inventory` feeds `file_inventory`.

---

18. Requirement and acceptance links

The artifact should summarize how requirements and acceptance obligations map into interface surfaces.

At minimum, it should support:

- requirement ID → endpoint/event
- acceptance criterion → endpoint proof/test target

This is not the full trace map, but it keeps the endpoint inventory grounded in actual kit obligations.

---

19. Generation implications section

The artifact should summarize what generation should expect from the API/interface layer.

Examples:

- total endpoint count
- total event/webhook count
- protected vs public endpoint count
- expected interface-driven file classes
- expected interface-driven file count contribution

This matters because it helps quantify whether the backend is being underbuilt.

---

20. Example endpoint entry


```
{
 "endpoint_id": "create_document",
 "name": "Create Document",
 "method": "POST",
 "path": "/documents",
 "purpose": "Creates a new document in the active workspace and returns the initialized
document summary.",
 "owner_module_id": "document_service",
 "request_contract_ref": "contract.document.create.request",
 "response_contract_ref": "contract.document.create.response",
 "auth_required": true,
 "permission_rules": [
   "workspace_member",
   "document_create_allowed"
 ],
 "source_requirement_refs": [
   "REQ-DOC-001"
 ],
 "source_artifact_refs": [
   "API-document-routes.md",
   "IMPL-document-lifecycle.md"
 ],
 "file_class_targets": [
   "route",
   "handler",
   "request_contract",
   "response_contract",
   "integration_test"
```

```
 ],
 "status": "active"
}
```


---

21. Example event entry


```
{
 "interface_event_id": "document_collaboration_update",
 "name": "Document Collaboration Update",
 "surface_type": "realtime_event",
 "purpose": "Broadcasts live collaboration state changes for an active document editing
session.",
 "owner_module_id": "collaboration_session_service",
 "trigger": "document_edit_applied",
 "payload_contract_ref": "event.document.collaboration.update",
 "auth_or_trust_boundary": "workspace_scoped_authenticated_session",
 "source_requirement_refs": [
   "REQ-COLLAB-002"
 ],
 "source_artifact_refs": [
   "API-realtime-collaboration.md",
   "ARCH-realtime-boundaries.md"
 ],
 "file_class_targets": [
   "event_contract",
   "gateway_handler",
   "realtime_service",
   "integration_test"
 ],
 "status": "active"
}
```


---

22. Anti-drift rules

`endpoint_inventory` should enforce these rules:

22.1 No source-less major interfaces

A major endpoint or event surface cannot exist without source linkage.

22.2 No generic CRUD collapse

If the kit implies workflow-specific routes or event surfaces, the inventory must preserve them.

22.3 No contract-less interfaces

A serious endpoint should not exist without request/response or payload contract linkage.

22.4 No security-blind interfaces

Protected, workspace-scoped, admin-scoped, or role-sensitive surfaces must be flagged as such.

22.5 No ownership ambiguity

Each interface surface should map to an owning module.

These rules stop the backend from turning into a shallow route stub layer.

---

23. Relationship to other projection artifacts

`endpoint_inventory` should connect directly to:

- `module_inventory`
- `entity_inventory`
- `requirement_trace_map`
- `verification_trace_map`
- `file_inventory`
- `repo_blueprint_seed`

It should be one of the main drivers of backend and shared-contract repo structure.

---

24. Completion rule

`endpoint_inventory` should only be considered complete when:

- all major HTTP/API and event surfaces implied by the kit are represented

- each surface has purpose, ownership, contract linkage, auth/permission structure, and source linkage
- the artifact is rich enough to drive interface-related file inventory derivation
- acceptance-relevant interface obligations are visible

If it cannot do that, it is incomplete.

---

25. Success definition

BPP-06 is successful when `endpoint_inventory` is defined as a structured projection artifact that decomposes Agent Kit truth into explicit HTTP, event, and webhook surfaces with methods, paths, ownership, contract linkage, permission rules, source references, and file-class implications, so the internal build system can derive a serious backend/API structure instead of collapsing the app into generic route scaffolding.

---

26. Short version for the doc

BPP-06 — Endpoint Inventory
`endpoint_inventory` is the Build Projection artifact that defines the interface surfaces the repo must contain. It maps Agent Kit API, workflow, implementation, security, and acceptance truth into explicit endpoints and event surfaces with ownership, contract references, permission rules, source linkage, and file-class implications. Its purpose is to preserve backend/API depth and interface structure before repo blueprinting and generation.

BPP-07 — Entity Inventory

BPP-07 — Entity Inventory

1. Purpose

Define the `entity_inventory` artifact as the projection artifact that turns canonical, data, API, implementation, and security truth from the Agent Kit into explicit domain-data structure.

BPP-07 exists because many build systems know an app “has data,” but they do not convert that into a disciplined entity model. The result is shallow schemas, weak relationships, missing validation, thin persistence structure, and backend logic that does not reflect the real product.

`entity_inventory` fixes that by making the domain-data surface explicit before the Repo Blueprint and File Inventory are derived.

---

2. Core role

`entity_inventory` is the artifact that defines:

- what entities exist
- what fields each entity has
- what relationships exist between entities
- what persistence and validation implications exist
- what lifecycle or state implications exist
- what modules and interfaces depend on those entities

It is the main build-facing decomposition of domain data into implementation-ready structure.

---

3. Main goal

The main goal of `entity_inventory` is to stop the build from reducing data truth into a few generic models.

Instead of:

- user model
- document model
- workspace model

the system should know:

- user identity entity
- workspace membership entity
- document entity
- document version entity
- comment entity
- AI suggestion entity
- permission assignment entity
- notification entity
- collaboration session entity

That is the level of decomposition this artifact should force.

---

4. Why this artifact is needed

A build system without an explicit entity inventory usually does one of two bad things:

4.1 Over-simplify

Collapse important domain distinctions into a few oversized models.

4.2 Over-improvise

Invent schemas and relationships during generation based on vague product prose.

`entity_inventory` prevents both by making the domain-data surface explicit before generation starts.

---

5. What this artifact represents

`entity_inventory` represents the **domain-data model the repository must implement**.

It is not:

- the raw data pack
- the full endpoint inventory
- the full file inventory
- the module inventory
- the migration files themselves

It is the data-layer projection artifact that sits between domain truth and file truth.

---

6. What this artifact consumes

The `entity_inventory` should be derived from these Agent Kit layers:

- canonical spec
- data pack
- implementation pack
- API pack
- work breakdown
- security pack where permissions or protected fields matter
- acceptance map where data obligations matter

The brief may influence it, but the entity inventory should never be brief-driven alone.

---

7. What this artifact produces

The `entity_inventory` should produce explicit definitions for:

- entities
- fields
- field types
- relationships
- constraints
- validation implications
- persistence implications
- lifecycle/state implications
- entity-to-module mapping
- entity-to-interface mapping
- entity-to-file-class implications

This makes it one of the strongest drivers of backend depth and shared-contract seriousness.

---

8. Required artifact structure

At minimum, `entity_inventory` should include these top-level sections:

1. artifact identity
2. entity list
3. relationship map
4. constraint and validation summary
5. persistence summary

6. module/interface links
7. generation implications

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `entity_inventory_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Entity model

Each entity entry should be explicit.

At minimum, each entity should include:

- `entity_id`
- `name`

- `purpose`
- `category`
- `fields`
- `relationships`
- `constraints`
- `state_model`
- `owner_module_ids`
- `interface_refs`
- `source_requirement_refs`
- `source_artifact_refs`
- `file_class_targets`
- `status`

Suggested `category` values:

- `core_domain`
- `supporting_domain`
- `join_or_membership`
- `audit_or_history`
- `workflow_or_session`
- `integration_or_external`

This helps separate primary business entities from supporting structures.

---

11. Field model

Each entity should contain explicit field definitions.

At minimum, each field should include:

- `field_id`
- `name`
- `type`
- `required`
- `nullable`
- `default_behavior`
- `sensitive`
- `validation_rules`

This prevents schemas from being reduced to vague property bags.

Suggested field `type` values might include:

- `string`
- `text`
- `boolean`
- `integer`
- `float`

- `datetime`
- `uuid`
- `json`
- `enum`
- `foreign_key`

---

12. Relationship rule

Every entity should declare its relationships.

At minimum, relationships should capture:

- related entity
- relationship type
- cardinality
- ownership direction

Suggested relationship types:

- `belongs_to`
- `has_many`
- `has_one`
- `many_to_many`

This helps derive:

- schema foreign keys

- repository logic
- route behavior
- shared contracts
- file inventory

---

13. Constraint rule

Each entity should declare meaningful constraints where applicable.

Examples:

- unique workspace slug
- unique email
- document version ordering
- permission uniqueness per member/workspace pair
- comment must belong to an existing document

This keeps entity modeling serious enough for real generation.

---

14. Validation rule

Each entity should carry validation implications, not just field names.

Examples:

- length constraints
- allowed enum values
- format rules

- cross-field requirements
- permission-sensitive field restrictions

This matters because many real repos need:

- validators
- DTO/request validation
- shared schema contracts
- form validation on the frontend

The entity inventory should help drive those.

---

15. State model rule

Where an entity has lifecycle behavior, that should be explicit.

Examples:

- document: draft → active → archived
- suggestion: pending → accepted → rejected
- invitation: created → accepted → expired
- collaboration session: open → active → closed

This matters because lifecycle truth often shapes:

- service logic
- permissions
- UI states
- tests

- acceptance proofs

---

16. Persistence rule

The artifact should summarize persistence implications.

At minimum, this should include:

- storage expectations
- migration expectations
- indexing implications where meaningful
- soft delete / archive behavior where relevant
- history/versioning implications where relevant

This does not need to become a full database spec, but it should preserve the build-relevant data shape.

---

17. Module and interface linkage

Each entity should identify:

- which modules own or operate on it
- which interfaces expose it or mutate it

This directly connects `entity_inventory` to:

- `module_inventory`
- `endpoint_inventory`
- `ui_inventory`

- `file_inventory`

Without that linkage, entities remain too isolated from build structure.

---

18. Source linkage rule

Every major entity should be attributable back to the Agent Kit.

At minimum:

- `source_requirement_refs`
- `source_artifact_refs`

That means no major entity should appear without source basis.

This is one of the main anti-drift rules for backend/data generation.

---

19. File-class target rule

Each entity should include what kinds of files it implies.

Examples:

- schema/model file
- migration file
- repository/data access file
- shared contract/type file
- validation file
- unit/integration test file

This is how `entity_inventory` feeds `file_inventory`.

---

20. Relationship map section

Beyond per-entity relationships, the artifact should provide a relationship summary.

This can include:

- entity graph
- major join entities
- version/history structures
- permission and membership structures

This helps preserve domain richness and prevents flattening.

---

21. Constraint and validation summary section

The artifact should summarize:

- total unique constraints
- total enum-backed fields
- total sensitive fields
- total validation-heavy entities

This helps quantify how serious the data layer is and later supports verification.

---

22. Generation implications section

The artifact should summarize what generation should expect from the data layer.

Examples:

- total entity count

- total field count
- total relationship count
- expected migration count
- expected model/repository/contract file classes
- expected entity-driven file count contribution

This matters because it helps quantify whether the backend and shared data structures are being underbuilt.

---

23. Example entity entry


```
{
 "entity_id": "document",
 "name": "Document",
 "purpose": "Represents a user-authored document within a workspace, including editable content and document-level metadata.",
 "category": "core_domain",
 "fields": [
   {
     "field_id": "document_id",
     "name": "id",
     "type": "uuid",
     "required": true,
     "nullable": false,
     "default_behavior": "generated",
     "sensitive": false,
     "validation_rules": []
   },
   {
     "field_id": "document_title",
     "name": "title",
     "type": "string",
     "required": true,
     "nullable": false,
     "default_behavior": "none",
     "sensitive": false,
     "validation_rules": ["min_length:1", "max_length:200"]
   }
```

```
],
"relationships": [
  {
    "related_entity_id": "workspace",
    "relationship_type": "belongs_to",
    "cardinality": "many_to_one",
    "ownership_direction": "document_to_workspace"
  },
  {
    "related_entity_id": "document_version",
    "relationship_type": "has_many",
    "cardinality": "one_to_many",
    "ownership_direction": "document_to_versions"
  }
],
"constraints": [
  "document_must_belong_to_workspace",
  "title_required"
],
"state_model": {
  "states": ["draft", "active", "archived"]
},
"owner_module_ids": [
  "document_service",
  "document_repository"
],
"interface_refs": [
  "create_document",
  "get_document",
  "update_document"
],
"source_requirement_refs": [
  "REQ-DOC-001",
  "REQ-DOC-002"
],
"source_artifact_refs": [
  "DATA-document-schema.md",
  "CAN-domain-concepts.md"
],
"file_class_targets": [
  "model",
  "migration",
  "repository",
  "shared_contract",
```

```
  "validation",
  "integration_test"
 ],
 "status": "active"
}
```

---

24. Anti-drift rules

`entity_inventory` should enforce these rules:

24.1 No source-less major entities

A major entity cannot exist without source linkage.

24.2 No domain flattening

If the kit implies distinct entities, the inventory must not collapse them into one generic record.

24.3 No relationship blindness

Entity relationships must not be omitted when they materially affect implementation.

24.4 No validation blindness

Important constraints and validation rules must not be dropped.

24.5 No permission-blind data modeling

Sensitive or permission-linked fields/relationships must be visible where relevant.

These rules stop the backend from turning into shallow model stubs.

---

25. Relationship to other projection artifacts

`entity_inventory` should connect directly to:

- `module_inventory`
- `endpoint_inventory`
- `requirement_trace_map`

- `verification_trace_map`
- `file_inventory`
- `repo_blueprint_seed`

It should be one of the main drivers of backend, data, and shared-contract repo structure.

---

26. Completion rule

`entity_inventory` should only be considered complete when:

- all major domain entities implied by the kit are represented
- each entity has fields, relationships, constraints, state/persistence implications, and source linkage
- the artifact is rich enough to drive entity-related file inventory derivation
- acceptance-relevant data obligations are visible

If it cannot do that, it is incomplete.

---

27. Success definition

BPP-07 is successful when `entity_inventory` is defined as a structured projection artifact that decomposes Agent Kit truth into explicit entities, fields, relationships, constraints, lifecycle implications, module/interface links, source references, and file-class implications, so the internal build system can derive a serious data and backend structure instead of collapsing the app into generic model scaffolding.

---

28. Short version for the doc

BPP-07 — Entity Inventory
`entity_inventory` is the Build Projection artifact that defines the domain-data structure the repo must contain. It maps Agent Kit canonical, data, implementation, API, security, and

acceptance truth into explicit entities with fields, relationships, constraints, lifecycle implications, source linkage, and file-class implications. Its purpose is to preserve domain depth and data seriousness before repo blueprinting and generation.

BPP-08 — File Inventory

BPP-08 — File Inventory

1. Purpose

Define the `file_inventory` artifact as the projection artifact that turns all prior Build Projection outputs into an explicit repository file target set.

BPP-08 exists because this is the artifact that finally answers the most operational build question:

What files must exist for this repo to be considered fully built?

Without `file_inventory`, the build agent stops when it feels done.
With `file_inventory`, the build agent stops when the planned repo is actually complete.

---

2. Core role

`file_inventory` is the artifact that defines:

- expected directories
- expected file classes
- expected concrete file targets
- file ownership by module/layer
- file purpose
- file traceability
- expected file counts by area
- total expected file count

It is the main materialization-target artifact for the generator.

---

3. Main goal

The main goal of `file_inventory` is to stop the internal build system from collapsing a rich Agent Kit into a small scaffold.

Instead of:

- “make the frontend”
- “make the backend”
- “make tests”

the system should know:

- exactly which file targets must exist
- which are required vs optional
- which module each file belongs to
- which requirements each file helps satisfy
- which verification obligations each file supports

That is what turns Build Mode into a real repo synthesizer.

---

4. Why this artifact is needed

A build system without an explicit file inventory usually does one of two bad things:

4.1 Underbuild

Stop after generating enough structure to look plausible.

4.2 Over-compress

Pack too much logic into too few files because no explicit target count or file map exists.

`file_inventory` prevents both by making the repo’s materialization target explicit before generation begins.

---

5. What this artifact represents

`file_inventory` represents the **concrete repository target set** implied by the Agent Kit and the Build Projection Pack.

It is not:

- the full repo itself
- the Repo Blueprint as a whole
- the build plan
- the verification report

It is the file-target projection artifact that sits between projected app structure and actual repo generation.

---

6. What this artifact consumes

The `file_inventory` should be derived from:

- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`
- `requirement_trace_map`
- `verification_trace_map`
- `repo_blueprint_seed`
- architecture and implementation implications from the Agent Kit
- build profile and repo shape constraints

This means it is not derived from one pack alone.
It is the convergence artifact of the Build Projection Pack.

---

7. What this artifact produces

The `file_inventory` should produce explicit definitions for:

- directories
- file groups
- file targets
- file class
- file purpose
- owning module
- related requirements
- related verification obligations
- required vs optional status
- expected counts by layer/area
- total expected file count

This makes it the strongest anti-underbuild artifact in the whole system.

---

8. Required artifact structure

At minimum, `file_inventory` should include these top-level sections:

1. artifact identity
2. directory inventory

3. file-group inventory
4. concrete file-target inventory
5. count summary
6. traceability summary
7. generation implications

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `file_inventory_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Directory inventory

The artifact should explicitly list the directories the repo must contain.

At minimum, each directory entry should include:

- `directory_id`
- `path`
- `purpose`
- `layer`
- `owning_scope`
- `required`

Examples:

- `src/app/pages`
- `src/app/components`
- `src/app/hooks`
- `src/server/routes`
- `src/server/services`
- `src/server/models`
- `src/shared/contracts`
- `src/shared/types`
- `tests/unit`
- `tests/integration`
- `docs`
- `scripts`

This stops structural collapse before file generation even starts.

---

11. File-group inventory

Before listing every file, the artifact should summarize file groups.

Each file group should include:

- `file_group_id`
- `name`
- `layer`
- `file_class`
- `owning_module_ids`
- `directory_path`
- `expected_count`
- `required`

Examples:

- backend route files
- backend service files
- backend validation files
- frontend page files
- frontend component files
- shared contract files
- migration files

- unit tests
- integration tests

This makes file planning inspectable at a higher level before looking at individual files.

---

12. Concrete file-target model

Each concrete file target should be explicit.

At minimum, each file entry should include:

- `file_id`
- `path`
- `file_class`
- `layer`
- `purpose`
- `owning_module_id`
- `related_entity_ids`
- `related_endpoint_ids`
- `related_screen_ids`
- `related_requirement_refs`
- `related_acceptance_refs`
- `required`
- `generation_status`

This is the core of the artifact.

---

13. File class rule

Each file must have an explicit class.

Suggested `file_class` values:

- `page`
- `layout`
- `component`
- `hook`
- `route`
- `handler`
- `service`
- `middleware`
- `model`
- `migration`
- `repository`
- `shared_contract`
- `shared_type`
- `validation`
- `event_contract`

- `test_unit`
- `test_integration`
- `test_e2e`
- `config`
- `script`
- `doc`

This helps:

- enforce repo structure
- estimate file counts
- drive generation ordering
- support verification

---

14. Purpose rule

Every file target should have a clear purpose statement.

Bad example:

- “document file”

Good example:

- “Exposes authenticated document CRUD and document-version routes for the server API.”

This matters because vague file targets lead to vague generation.

---

15. Ownership rule

Each file should map to an owning module or owning structural scope.

Examples:

- `document_service`
- `workspace_dashboard_ui`
- `auth_guard_middleware`
- `document_contracts`

This keeps file generation anchored to projected system structure rather than ad hoc filenames.

---

16. Traceability rule

Each file target should link back to:

- relevant requirements
- relevant acceptance rules where applicable
- relevant projection artifacts

That means the system should be able to answer:

- why does this file exist
- what does it satisfy
- what source truth caused it to be projected

This is one of the most important anti-drift rules in the whole system.

---

17. Required vs optional rule

Each file target should declare whether it is:

- required
- optional
- profile-conditional

This matters because not every file is equal.
The generator must know which targets are mandatory for completion.

For the first version, you can simplify to:

- `required: true|false`

But the concept should remain.

---

18. Generation status rule

Even though the inventory is produced before generation, it should include a field the generator can later update or compare against.

Suggested values:

- `planned`
- `generated`
- `failed`
- `skipped`

This makes `file_inventory` useful across:

- planning
- generation

- verification
- failure analysis

---

19. Count summary section

This section is critical.

The artifact should summarize:

- total expected directories
- total expected file groups
- total expected files
- expected file count by layer
- expected file count by file class
- expected file count by subsystem/module

This is the section that directly exposes whether the build is trying to produce:

- 120 files
- 400 files
- 900 files

instead of hiding the true intended repo size.

---

20. Traceability summary section

The artifact should summarize:

- total files mapped to at least one requirement

- total files mapped to at least one acceptance rule
- total files with module ownership
- total files without source trace
- total required files vs optional files

This is important because a weak file inventory often has poor traceability.

---

21. Generation implications section

The artifact should summarize what generation is expected to do.

Examples:

- minimum required generated file count
- required file completion threshold
- required directory completion threshold
- required file-class completion thresholds
- verification-critical file classes

This helps the Build Runner and Verifier understand what “complete enough” means.

---

22. Example directory entry

```
{
 "directory_id": "server_services_dir",
 "path": "src/server/services",
 "purpose": "Application service layer for backend business logic modules.",
 "layer": "backend",
 "owning_scope": "backend_api",
 "required": true
}
```

---

23. Example file-group entry

```
{
 "file_group_id": "backend_service_files",
 "name": "Backend Service Files",
 "layer": "backend",
 "file_class": "service",
 "owning_module_ids": [
   "document_service",
   "workspace_service",
   "auth_service"
 ],
 "directory_path": "src/server/services",
 "expected_count": 12,
 "required": true
}
```

---

24. Example concrete file entry

```
{
 "file_id": "document_service_file",
 "path": "src/server/services/documentService.ts",
 "file_class": "service",
 "layer": "backend",
 "purpose": "Implements document lifecycle operations and document-level business rules.",
 "owning_module_id": "document_service",
 "related_entity_ids": ["document", "document_version"],
 "related_endpoint_ids": ["create_document", "get_document", "update_document"],
 "related_screen_ids": [],
 "related_requirement_refs": ["REQ-DOC-001", "REQ-DOC-002"],
 "related_acceptance_refs": ["ACC-DOC-EDIT-001"],
 "required": true,
 "generation_status": "planned"
}
```

---

25. Anti-drift rules

`file_inventory` should enforce these rules:

25.1 No source-less required files

A required file should not exist without traceability to projected kit truth.

25.2 No module-less serious files

Serious implementation files should have clear module ownership.

25.3 No generic bucket collapse

If the Build Projection Pack implies distinct files, the inventory must not collapse them into one broad placeholder.

25.4 No hidden file-count compression

The inventory must expose expected file volume explicitly.

25.5 No verification-blind file planning

Files that are critical for proof and completion must be marked as such through trace linkage.

These rules directly address the weak-build problem you found.

---

26. Relationship to other projection artifacts

`file_inventory` should connect directly to:

- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`
- `requirement_trace_map`
- `verification_trace_map`
- `repo_blueprint_seed`

It should be one of the immediate drivers of:

- `repo_blueprint`

- `build_plan`
- generator target loops
- verification completeness checks

---

27. Relationship to Build Mode

This artifact should become one of the main build gates.

In practice:

- Build Mode should derive the file inventory
- Build Mode should compare generated output against it
- Build completion should depend heavily on file inventory coverage

So this artifact is not just documentation.
It should be operationally binding.

---

28. Completion rule

`file_inventory` should only be considered complete when:

- the expected directory structure is explicit
- the major file groups are explicit
- the concrete file targets are explicit enough to drive generation
- required file counts and total expected file count are explicit
- files are traceable to projected kit truth

If it cannot do that, it is incomplete.

---

29. Success definition

BPP-08 is successful when `file_inventory` is defined as a structured projection artifact that converts module, UI, interface, entity, and trace projections into explicit directories, file groups, concrete file targets, count expectations, and traceable file-level obligations, so the internal build system can generate to a real completion target instead of stopping at a shallow scaffold.

---

30. Short version for the doc

BPP-08 — File Inventory

`file_inventory` is the Build Projection artifact that defines the concrete repository target set the internal build system must materialize. It maps projected modules, UI surfaces, interfaces, entities, and trace obligations into explicit directories, file groups, file targets, ownership, traceability, and expected file counts. Its purpose is to make repo completion measurable and to prevent underbuilt outputs caused by shallow or compressed generation.

BPP-09 — Requirement Trace Map

BPP-09 — Requirement Trace Map

1. Purpose

Define the `requirement_trace_map` artifact as the projection artifact that connects Agent Kit requirements and work items to the concrete implementation units that will satisfy them.

BPP-09 exists because many build systems **read requirements but lose them during generation**. Requirements get summarized into features, but they are not tied to specific modules, files, endpoints, or UI surfaces.

The `requirement_trace_map` prevents that loss by explicitly mapping:

requirement → module → interface/UI/data → file targets

This artifact proves the build is **driven by the Agent Kit instead of loosely inspired by it**.

---

2. Core role

`requirement_trace_map` defines how each requirement from the Agent Kit is implemented within the projected system structure.

It answers questions like:

- which module implements a requirement
- which endpoint exposes it
- which UI surface supports it
- which entity stores its data
- which files will contain the implementation

This artifact forms the **coverage bridge between the Agent Kit and the repository**.

---

3. Main goal

The main goal of `requirement_trace_map` is to eliminate **requirement loss during build translation**.

Without this artifact:

Requirement → feature summary → vague implementation

With this artifact:

Requirement
→ module(s)
→ endpoint(s)
→ UI surface(s)
→ entity/entities
→ file targets

This ensures that every requirement has a **clear implementation path**.

---

4. Why this artifact is needed

Without a requirement trace map, builds typically fail in two ways.

4.1 Requirement collapse

Multiple requirements collapse into one generic implementation.

Example:

REQ-EDIT-001
REQ-EDIT-002
REQ-EDIT-003
→ one generic "editor page"

4.2 Requirement loss

Some requirements disappear completely during generation because they are not tied to concrete structures.

`requirement_trace_map` prevents both by making requirement coverage explicit.

---

5. What this artifact represents

`requirement_trace_map` represents the **implementation coverage of the Agent Kit**.

It does not define new requirements.
It simply projects existing requirements into system structure.

It sits between:

Agent Kit requirements
↓
Requirement Trace Map
↓
Implementation structure
↓
File inventory
↓
Generated repo

---

6. What this artifact consumes

The `requirement_trace_map` should be derived from:

- canonical spec
- work breakdown
- acceptance map
- implementation pack
- design pack
- API pack
- data pack
- security pack
- module inventory
- UI inventory
- endpoint inventory

- entity inventory

This artifact is where **all projection layers converge**.

---

7. What this artifact produces

The `requirement_trace_map` produces structured mappings between:

- requirements
- modules
- UI surfaces
- endpoints
- entities
- file targets

It also produces:

- requirement coverage summaries
- implementation density metrics
- uncovered requirement flags

---

8. Required artifact structure

At minimum, `requirement_trace_map` should include:

1. artifact identity
2. requirement list
3. requirement-to-implementation mappings

4. requirement coverage summary
5. uncovered requirement report
6. generation implications

---

9. Top-level identity fields

At minimum:

- `requirement_trace_map_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

---

10. Requirement model

Each requirement entry should contain:

- `requirement_id`
- `name`
- `description`
- `source_artifact_ref`

- `priority`
- `status`

Suggested `priority` values:

- `core`
- `important`
- `supporting`
- `optional`

---

11. Implementation mapping model

Each requirement should map to one or more implementation targets.

Implementation targets include:

- modules
- UI screens
- endpoints
- entities
- files

This makes the trace map the **connective tissue between all projection artifacts**.

---

12. Mapping fields

Each mapping entry should include:

- `requirement_id`
- `module_ids`
- `screen_ids`
- `endpoint_ids`
- `entity_ids`
- `file_ids`
- `verification_refs`
- `status`

Suggested `status` values:

- `planned`
- `mapped`
- `generated`
- `verified`

---

13. Requirement coverage rule

Each requirement should map to **at least one implementation unit**.

Examples:

REQ-DOC-001
→ document_service
→ create_document endpoint
→ editor_workspace screen
→ document entity

→ documentService.ts

If a requirement cannot map to implementation units, the system should flag it.

---

14. Multi-layer mapping rule

Serious requirements often span multiple layers.

Example:

REQ-COLLAB-002

May map to:

module_ids:
- collaboration_service

endpoint_ids:
- collaboration_update_event

screen_ids:
- editor_workspace

entity_ids:
- collaboration_session

The trace map should preserve that complexity.

---

15. Requirement-to-file linkage

Every mapped requirement should ultimately link to **file targets**.

Example:

REQ-DOC-001
→ src/server/services/documentService.ts
→ src/server/routes/documentRoutes.ts
→ src/app/pages/Editor.tsx

This ensures the requirement actually becomes code.

---

16. Source linkage rule

Every requirement entry must reference its origin in the Agent Kit.

Example:

```
source_artifact_ref:
- CAN-document-behavior.md
- IMPL-editor-workflow.md
```

This preserves traceability from repo → kit.

---

17. Coverage summary section

The artifact should summarize:

- total requirements
- mapped requirements
- unmapped requirements
- requirements with module coverage
- requirements with file coverage

Example:

```
total_requirements: 126
mapped_requirements: 124
unmapped_requirements: 2
```

---

18. Uncovered requirement report

If requirements cannot be mapped to implementation units, they must be flagged.

Example reasons:

- missing module projection
- missing endpoint projection

- incomplete UI projection
- missing entity structure

This helps identify gaps in the projection pack.

---

19. Generation implications

The artifact should summarize:

- minimum implementation units required for requirement coverage
- file targets driven by requirements
- requirement density across modules
- requirement coverage by subsystem

This helps the build system detect if generation output is incomplete.

---

20. Example trace entry


```
{
 "requirement_id": "REQ-DOC-001",
 "name": "Create Document",
 "description": "Users must be able to create new documents within a workspace.",
 "module_ids": [
   "document_service"
 ],
 "screen_ids": [
   "workspace_dashboard"
 ],
 "endpoint_ids": [
   "create_document"
 ],
 "entity_ids": [
   "document"
 ],
 "file_ids": [
   "document_service_file",
```

```
  "document_routes_file"
 ],
 "verification_refs": [
  "ACC-DOC-CREATE-001"
 ],
 "status": "mapped"
}
```


---

21. Anti-drift rules

`requirement_trace_map` should enforce:

No unmapped core requirements

All core requirements must map to implementation units.

No implementation without requirement basis

Major modules or endpoints should link back to requirements.

No shallow mapping

Requirements should map beyond a single module where appropriate.

No trace loss

Requirements should remain visible through all projection layers.

---

22. Relationship to other projection artifacts

`requirement_trace_map` connects directly to:

- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`

- `file_inventory`
- `verification_trace_map`

It is one of the most important **integrity artifacts** in the projection system.

---

23. Completion rule

The trace map is complete when:

- every core requirement maps to implementation units
- file targets exist for mapped requirements
- no major requirement remains unmapped
- the artifact proves requirement coverage across system structure

---

24. Success definition

BPP-09 is successful when `requirement_trace_map` explicitly connects Agent Kit requirements to modules, UI surfaces, endpoints, entities, and file targets, ensuring the internal build system implements the full scope of the kit instead of compressing it into shallow scaffolding.

---

25. Short version for the doc

BPP-09 — Requirement Trace Map
`requirement_trace_map` connects Agent Kit requirements to the projected system structure. It maps requirements to modules, UI surfaces, endpoints, entities, and file targets to ensure the build system preserves requirement coverage during repo generation and prevents requirement loss or collapse.

BPP-10 — Verification Trace Map

BPP-10 — Verification Trace Map

1. Purpose

Define the `verification_trace_map` artifact as the projection artifact that connects Agent Kit acceptance and proof obligations to the concrete verification surfaces the repo must contain.

BPP-10 exists because many systems can generate code from requirements, but they do not preserve **how that code will be proven valid**. Acceptance criteria stay in documents, while tests, checks, and proof artifacts are generated loosely or incompletely.

The `verification_trace_map` fixes that by explicitly mapping:

acceptance criterion → proof target → test/check → file/artifact

This artifact proves the build is not only implementing the kit, but also preserving the kit's completion logic.

---

2. Core role

`verification_trace_map` defines how each acceptance condition, proof obligation, or verification expectation from the Agent Kit is translated into concrete validation surfaces.

It answers questions like:

- what test proves this acceptance criterion
- what verification category covers this obligation
- what file or artifact contains that proof
- what module or interface is being validated
- what check must pass before the build can count as complete

It is the build-facing bridge between **acceptance truth** and **verification execution**.

---

3. Main goal

The main goal of `verification_trace_map` is to eliminate **acceptance loss during build translation**.

Without it:

acceptance criteria → general “tests will exist”

With it:

acceptance criterion
→ proof target
→ verification category
→ test/check/artifact
→ file target

That ensures completion remains governed by the Agent Kit instead of becoming vague post-build validation.

---

4. Why this artifact is needed

Without an explicit verification trace map, builds usually fail in two ways.

4.1 Acceptance collapse

Many acceptance rules are reduced to one generic test suite or one vague “verification passed” result.

4.2 Proof loss

Some obligations never become:

- tests
- checks
- scans
- verification artifacts

So the build can look complete while not proving what the kit required.

`verification_trace_map` prevents both by making proof surfaces explicit before generation and verification.

---

5. What this artifact represents

`verification_trace_map` represents the **proof coverage of the Agent Kit**.

It does not define new acceptance criteria.
It projects existing acceptance, gate, and proof obligations into executable verification structure.

It sits between:

Agent Kit acceptance / proof truth
↓
Verification Trace Map
↓
test/check/artifact targets
↓
verification execution
↓
build completion decision

---

6. What this artifact consumes

The `verification_trace_map` should be derived from:

- acceptance map
- work breakdown
- canonical spec
- implementation pack
- design pack
- API pack
- security pack
- module inventory
- UI inventory
- endpoint_inventory
- entity_inventory
- file_inventory
- requirement_trace_map

This is where acceptance and proof obligations converge with projected implementation structure.

---

7. What this artifact produces

The `verification_trace_map` produces structured mappings between:

- acceptance criteria

- proof obligations
- verification categories
- test targets
- check targets
- proof artifacts
- file targets

It also produces:

- acceptance coverage summaries
- unmapped acceptance flags
- proof-density summaries by subsystem/layer

---

8. Required artifact structure

At minimum, `verification_trace_map` should include:

1. artifact identity
2. acceptance / proof obligation list
3. obligation-to-verification mappings
4. verification coverage summary
5. uncovered obligation report
6. generation and completion implications

---

9. Top-level identity fields

At minimum:

- `verification_trace_map_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

---

10. Acceptance obligation model

Each acceptance or proof entry should contain:

- `acceptance_id`
- `name`
- `description`
- `source_artifact_ref`
- `priority`
- `proof_type`
- `status`

Suggested `priority` values:

- `core`
- `important`
- `supporting`
- `optional`

Suggested `proof_type` values:

- `unit_behavior`
- `integration_behavior`
- `ui_behavior`
- `security_control`
- `data_integrity`
- `structural_check`
- `artifact_presence`
- `command_execution`

This keeps proof obligations typed and build-usable.

---

11. Verification mapping model

Each acceptance or proof obligation should map to one or more verification targets.

Verification targets include:

- verification categories
- test files
- command checks
- scans
- manifest/report artifacts

- validated modules/endpoints/screens/entities

This is what turns acceptance into operational proof.

---

12. Mapping fields

Each mapping entry should include:

- `acceptance_id`
- `verification_category_ids`
- `test_file_ids`
- `check_ids`
- `artifact_ids`
- `validated_module_ids`
- `validated_endpoint_ids`
- `validated_screen_ids`
- `validated_entity_ids`
- `status`

Suggested `status` values:

- `planned`
- `mapped`
- `generated`
- `verified`

---

13. Acceptance coverage rule

Each core acceptance obligation should map to at least one proof surface.

Examples:

ACC-DOC-EDIT-001
→ verification_category: required_command_execution
→ test_file: tests/integration/documentEditing.test.ts
→ validated_module: document_service
→ validated_screen: editor_workspace

If a core acceptance rule cannot map to verification surfaces, the system should flag it.

---

14. Multi-surface proof rule

Serious obligations often require more than one proof surface.

Example:

ACC-COLLAB-002

May require:

verification_category_ids:
- ui_behavior
- integration_behavior

test_file_ids:
- editorCollaboration.integration.test.ts
- realtimePresence.ui.test.ts

validated_module_ids:
- collaboration_session_service

validated_screen_ids:
- editor_workspace

validated_endpoint_ids:
- document_collaboration_update

The trace map should preserve that multi-surface proof structure.

---

15. Acceptance-to-file linkage

Every mapped acceptance rule should ultimately link to:

- one or more test/check/proof files
- and, where appropriate, one or more non-test artifacts such as manifests or reports

This ensures acceptance becomes something the system can actually execute and record.

---

16. Source linkage rule

Every acceptance entry must reference its origin in the Agent Kit.

Example:

- acceptance map file
- architecture verification note
- security proof requirement
- implementation proof rule

This preserves traceability from verification → build → kit.

---

17. Verification category linkage

Each acceptance obligation should map into one or more Build Mode verification categories.

Examples:

- required output presence
- repo structure correctness
- placeholder scan
- manifest completeness
- required command execution
- export readiness
- security-control verification
- integration proof

This directly ties the projection layer to Build Mode verification.

---

18. Coverage summary section

The artifact should summarize:

- total acceptance obligations
- mapped obligations
- unmapped obligations
- obligations with test coverage
- obligations with command/check coverage
- obligations with artifact/report coverage

Example:

total_acceptance_obligations: 94
mapped_obligations: 92
unmapped_obligations: 2

---

19. Uncovered obligation report

If acceptance or proof obligations cannot be mapped to verification surfaces, they must be flagged.

Example reasons:

- missing module projection
- missing UI projection
- no derived proof artifact
- no suitable verification category
- file inventory gap

This helps identify proof holes before generation completes.

---

20. Generation and completion implications

The artifact should summarize:

- minimum test/check/proof files required
- required proof artifacts for completion
- verification-critical modules
- acceptance coverage by subsystem
- acceptance coverage by file class
- acceptance coverage thresholds required for pass

This helps Build Mode treat verification as a first-class completion gate.

---

21. Example trace entry

{
  "acceptance_id": "ACC-DOC-EDIT-001",
  "name": "Document Editing Persists Correctly",
  "description": "Document changes must be saved and reloaded accurately within the active workspace.",
  "verification_category_ids": [

```
    "required_command_execution",
    "integration_behavior"
  ],
  "test_file_ids": [
    "document_editing_integration_test",
    "document_persistence_unit_test"
  ],
  "check_ids": [
    "typecheck",
    "integration_test_run"
  ],
  "artifact_ids": [
    "verification_report"
  ],
  "validated_module_ids": [
    "document_service",
    "document_repository"
  ],
  "validated_endpoint_ids": [
    "update_document",
    "get_document"
  ],
  "validated_screen_ids": [
    "editor_workspace"
  ],
  "validated_entity_ids": [
    "document",
    "document_version"
  ],
  "status": "mapped"
}
```


---

22. Anti-drift rules

`verification_trace_map` should enforce:

No unmapped core acceptance obligations

All core acceptance rules must map to proof surfaces.

No proof without source basis

Major tests/checks/proof artifacts should link back to acceptance obligations.

No courtesy verification

An obligation cannot be considered covered without an actual mapped proof target.

No verification compression

Distinct obligations should not be collapsed into one vague “general test” unless explicitly justified.

No completion without proof coverage

If acceptance cannot be traced to proof artifacts, the build should not count as fully build-ready.

These rules stop the build from being “implemented” but not “provable.”

---

23. Relationship to other projection artifacts

`verification_trace_map` connects directly to:

- `requirement_trace_map`
- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`
- `file_inventory`

It is the proof-side integrity companion to the requirement trace map.

---

24. Completion rule

The verification trace map is complete when:

- every core acceptance obligation maps to proof targets
- test/check/artifact targets exist for mapped obligations
- no major acceptance rule remains unmapped
- the artifact proves acceptance coverage across projected implementation structure

---

25. Success definition

BPP-10 is successful when `verification_trace_map` explicitly connects Agent Kit acceptance and proof obligations to verification categories, tests, checks, artifacts, and validated implementation surfaces, ensuring the internal build system preserves proof coverage during repo generation and does not detach completion logic from the kit.

---

26. Short version for the doc

BPP-10 — Verification Trace Map
`verification_trace_map` connects Agent Kit acceptance and proof obligations to the projected verification structure. It maps acceptance criteria to verification categories, tests, checks, artifacts, and validated modules, UI surfaces, endpoints, entities, and file targets so the build system preserves proof coverage and completion logic during repo generation.

Say `next` and I'll do **BPP-11 — Repo Blueprint Seed**.

BPP-11 — Repo Blueprint Seed

BPP-11 — Repo Blueprint Seed

1. Purpose

Define the `repo_blueprint_seed` artifact as the projection artifact that consolidates all Build Projection outputs into the structural seed for the full Repo Blueprint.

BPP-11 exists because the Build Projection Pack produces many important inventories and maps, but the system still needs one artifact that gathers that projected truth into a repo-structure-ready form.

The `repo_blueprint_seed` is that bridge.

It is the artifact that says:

- this is the repo type
- this is the repo shape
- these are the top-level areas
- these are the subsystem placements
- these are the structural boundaries
- these are the major generation implications

It is not yet the full Repo Blueprint, but it is the direct input that should make deriving the full Repo Blueprint deterministic.

---

2. Core role

`repo_blueprint_seed` defines the structural repo implications of the Build Projection Pack.

It answers questions like:

- what overall repo form is this build targeting
- how are the major subsystems placed in the repo

- what are the top-level packages/folders
- what layer boundaries exist
- what shared/common areas are required
- what file inventory and module structure imply about repo layout

It is the structural convergence artifact for the Build Projection Pack.

---

3. Main goal

The main goal of `repo_blueprint_seed` is to stop the system from jumping directly from projected inventories to freeform repo planning.

Without it:

projection artifacts → planner guesses repo structure

With it:

projection artifacts
→ repo_blueprint_seed
→ full repo_blueprint
→ build_plan

That preserves structure and reduces planner drift.

---

4. Why this artifact is needed

Without a repo blueprint seed, the system usually fails in one of two ways.

4.1 Structural compression

The planner sees modules, endpoints, UI, and files, but still compresses them into a simpler repo than the kit implies.

4.2 Structural improvisation

The planner invents repo organization patterns that were never made explicit by the projection layer.

`repo_blueprint_seed` prevents both by forcing projected structure into a repo-facing seed before full blueprint derivation begins.

---

5. What this artifact represents

`repo_blueprint_seed` represents the **structural repo truth implied by the Build Projection Pack**.

It is not:

- the full Repo Blueprint
- the build plan
- the file inventory
- the module inventory
- the final generated repo

It is the structural seed artifact that sits between projection outputs and the full Repo Blueprint.

---

6. What this artifact consumes

The `repo_blueprint_seed` should be derived from:

- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`
- `file_inventory`
- `requirement_trace_map`

- `verification_trace_map`
- build profile constraints
- repo shape implications from the brief/canonical/architecture layers

This means it is the artifact where all major projection outputs converge into a single structural interpretation.

---

7. What this artifact produces

The `repo_blueprint_seed` should produce explicit definitions for:

- repo type
- repo shape
- top-level structural areas
- layer placement
- subsystem placement
- package/folder placement hints
- structural dependency direction
- build- and verification-relevant repo implications

This makes it the direct precursor to the full Repo Blueprint.

---

8. Required artifact structure

At minimum, `repo_blueprint_seed` should include:

1. artifact identity
2. repo identity and type

3. repo shape
4. top-level structure
5. subsystem placement map
6. layer and dependency rules
7. generation implications
8. blueprint derivation summary

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `repo_blueprint_seed_id`
- `run_id`
- `kit_ref`
- `source_refs`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Repo identity model

The artifact should explicitly define the high-level repo identity.

At minimum:

- `app_name`
- `app_type`
- `repo_type`
- `repo_shape`
- `build_profile_ref`

Suggested `repo_type` values:

- `web_app`
- `mobile_app`
- `api_service`
- `fullstack_monorepo`

Suggested `repo_shape` values:

- `single_app`
- `client_server_split`
- `monorepo_with_shared`
- `mobile_plus_backend`

This is the first place where the system should lock the overall repository form.

---

11. Top-level structure model

The seed should explicitly define the top-level repo areas.

At minimum, each top-level structure entry should include:

- `structure_id`
- `path`
- `purpose`
- `layer_scope`
- `required`

Examples:

- `src/app`
- `src/server`
- `src/shared`
- `tests`
- `docs`
- `scripts`
- `config`

This is not yet the full directory tree, but it should set the main structural pillars.

---

12. Subsystem placement map

The artifact should define where each subsystem belongs in the repo.

At minimum, each subsystem placement entry should include:

- `subsystem_id`
- `placement_root`
- `layer`
- `related_module_ids`
- `notes`

Examples:

- `frontend_app` → `src/app`
- `backend_api` → `src/server`
- `shared_contracts` → `src/shared/contracts`
- `realtime_collaboration` → `src/server/realtime`
- `auth_and_access` → `src/server/auth`

This is where projected modules become repo placement truth.

---

13. Layer rule set

The artifact should define structural layer rules.

Examples:

- frontend code must live under app-layer roots
- backend services must not live inside frontend paths
- shared contracts/types must be structurally separate from server internals
- tests should be separated by type or scope

- migrations should live in data-layer paths

This matters because repo shape is more than folder names; it is also placement discipline.

---

14. Dependency direction rule

The seed should explicitly describe allowed structural dependency direction.

Examples:

- frontend may depend on shared contracts, not backend internals
- routes may depend on services, not UI modules
- services may depend on repositories and shared contracts
- shared types/contracts should not depend on frontend pages or backend handlers
- tests may depend on implementation, not the reverse

This preserves architecture during blueprint derivation.

---

15. Structural implications from inventories

The `repo_blueprint_seed` should summarize how projection artifacts influence structure.

Examples:

- module inventory implies 6 backend module groups
- UI inventory implies dedicated page, layout, and component-family zones
- endpoint inventory implies route, handler, and contract placement
- entity inventory implies models, migrations, validation, repository structure
- file inventory implies explicit counts and subdirectories by layer

This is how the seed proves it is truly derived from the Build Projection Pack.

---

16. Build-supporting areas

The seed should also include structural implications for supporting areas such as:

- test roots
- config roots
- docs roots
- script roots
- verification artifact support paths where relevant to the repo
- migration/seed/data helper paths

These are often omitted in weak repo designs, leading to underbuilt outputs.

---

17. Generation implications section

The artifact should summarize what generation should expect from the structural seed.

Examples:

- expected top-level directory count
- expected subsystem roots
- expected layer separation rules
- expected shared/common roots
- expected blueprint complexity level
- total expected file count inherited from `file_inventory`

This matters because it helps the planner understand the scale and seriousness of the repo before full blueprint derivation.

---

18. Blueprint derivation summary

The artifact should explicitly state what the full Repo Blueprint must still derive from this seed.

Examples:

- detailed directory tree
- package-level structure
- concrete file placement matrix
- exact structural validation rules
- detailed blueprint-to-file links

This keeps the distinction clear:

- `repo_blueprint_seed` = structural seed
- `repo_blueprint` = full structural build authority

---

19. Example seed structure

A simplified example:


```
{
 "repo_blueprint_seed_id": "RBPS-0001",
 "run_id": "RUN-0001",
 "kit_ref": "KIT-0001",
 "source_refs": [
   "module_inventory",
   "ui_inventory",
   "endpoint_inventory",
   "entity_inventory",
   "file_inventory"
 ],
 "app_name": "Atizay",
 "app_type": "saas",
```

"repo_type": "fullstack_monorepo",
"repo_shape": "monorepo_with_shared",
"build_profile_ref": "PROFILE-0001",
"top_level_structure": [
  {
    "structure_id": "app_root",
    "path": "src/app",
    "purpose": "Frontend application layer",
    "layer_scope": ["frontend"],
    "required": true
  },
  {
    "structure_id": "server_root",
    "path": "src/server",
    "purpose": "Backend application layer",
    "layer_scope": ["backend", "security", "data"],
    "required": true
  },
  {
    "structure_id": "shared_root",
    "path": "src/shared",
    "purpose": "Shared contracts and types",
    "layer_scope": ["shared"],
    "required": true
  }
],
"subsystem_placement_map": [
  {
    "subsystem_id": "frontend_app",
    "placement_root": "src/app",
    "layer": "frontend",
    "related_module_ids": ["workspace_dashboard_ui", "editor_workspace_ui"]
  },
  {
    "subsystem_id": "backend_api",
    "placement_root": "src/server",
    "layer": "backend",
    "related_module_ids": ["document_service", "workspace_service", "auth_service"]
  }
],
"dependency_direction_rules": [
  "frontend_depends_on_shared_only",
  "routes_depend_on_services_not_repositories_directly",
  "shared_has_no_dependency_on_app_or_server"

```
 ],
 "generation_implications": {
   "expected_top_level_roots": 6,
   "expected_subsystem_roots": 8,
   "expected_total_file_count": 720
 },
 "created_at": "2026-03-06T12:04:00Z",
 "updated_at": "2026-03-06T12:04:00Z",
 "status": "active"
}
```


---

20. Anti-drift rules

`repo_blueprint_seed` should enforce these rules:

20.1 No repo shape without projection basis

Repo type and shape must come from projection truth, not planner preference.

20.2 No hidden structural compression

If the projection artifacts imply multiple subsystem roots, the seed must preserve them.

20.3 No layer ambiguity

The seed must clearly distinguish frontend, backend, shared, data, security, and test/supporting structure where relevant.

20.4 No dependency-direction loss

If architecture and projection artifacts imply structural dependency constraints, the seed must preserve them.

20.5 No support-area omission

Tests, configs, scripts, docs, migrations, and other serious repo support areas must not disappear when the seed is formed.

These rules stop the planner from turning the projection pack into a weaker repo form.

---

21. Relationship to the full Repo Blueprint

This is the key relationship:

- `repo_blueprint_seed` is produced by the Build Projection Pack
- `repo_blueprint` is produced from the seed plus build-profile and structural derivation rules

So the seed should be strong enough that the full Repo Blueprint becomes mostly an act of elaboration, not reinvention.

That is the real value of this artifact.

---

22. Relationship to Build Mode

The `repo_blueprint_seed` should be one of the key entry artifacts for the Build Mode planning stage.

In practice:

- Build Projection Pack finishes
- `repo_blueprint_seed` exists
- planner derives full `repo_blueprint`
- planner derives `build_plan`
- generator materializes repo against those

So this artifact is one of the main points where the Agent Kit turns into build structure.

---

23. Completion rule

`repo_blueprint_seed` should only be considered complete when:

- repo type and shape are explicit
- top-level structure is explicit

- subsystem placement is explicit
- layer and dependency rules are explicit
- the seed clearly reflects the projection artifacts and is sufficient to derive a full Repo Blueprint without major guessing

If it cannot do that, it is incomplete.

---

24. Success definition

BPP-11 is successful when `repo_blueprint_seed` is defined as a structured projection artifact that consolidates the Build Projection Pack into explicit repo type, repo shape, top-level structure, subsystem placement, layer rules, and dependency-direction rules, so the full Repo Blueprint can be derived as a deterministic elaboration of projected kit truth rather than a planner-driven reinvention.

---

25. Short version for the doc

BPP-11 — Repo Blueprint Seed
`repo_blueprint_seed` is the Build Projection artifact that consolidates module, UI, endpoint, entity, file, and trace projections into the structural seed for the full Repo Blueprint. It defines repo type, repo shape, top-level structural areas, subsystem placement, and layer/dependency rules so the internal build system can derive repo structure deterministically from the Agent Kit instead of improvising it.

BPP-12 — Kit Extraction Report

BPP-12 — Kit Extraction Report

1. Purpose

Define the `kit_extraction_report` artifact as the proof artifact that shows whether the Agent Kit was actually extracted deeply enough to support a real build.

BPP-12 exists because the internal build system should not be allowed to say “the kit was used” without proving:

- what kit files were consumed
- what kit layers were extracted
- what projection artifacts were produced
- what structural build truth was derived
- what gaps remain
- whether build may proceed

This artifact is the **proof of kit consumption**.

---

2. Core role

`kit_extraction_report` is the artifact that records the result of Agent Kit extraction before repo generation begins.

It answers questions like:

- how many kit files were consumed
- which source layers were extracted
- which projection artifacts were produced
- whether extraction was complete enough to continue

- what unresolved extraction gaps remain
- what file-count expectation was derived

It is the gate artifact that prevents Build Mode from starting on shallow kit usage.

---

3. Main goal

The main goal of `kit_extraction_report` is to prevent the internal build system from doing this:

read brief
read feature list
infer stack
start generating repo

Instead, it should force this:

load kit
extract required source layers
produce projection artifacts
derive repo/file implications
prove extraction completeness
only then allow build planning/generation

That is the reason this artifact exists.

---

4. Why this artifact is needed

Without an explicit extraction report, the system usually fails in two ways.

4.1 False extraction confidence

The system claims the kit was consumed, but actually only used:

- the brief
- feature names
- maybe a few summary docs

4.2 Invisible extraction loss

Large parts of the kit are ignored, but there is no artifact showing what was not used or why.

`kit_extraction_report` prevents both by making extraction measurable, auditable, and gateable.

---

5. What this artifact represents

`kit_extraction_report` represents the **completeness and quality of Agent Kit extraction** for a build run.

It is not:

- the Agent Kit itself
- the Build Projection Pack itself
- the Repo Blueprint
- the Build Plan
- the final verification report

It is the proof artifact that says whether the kit was extracted deeply enough to justify those downstream steps.

---

6. What this artifact consumes

The `kit_extraction_report` should be derived from:

- the Agent Kit manifest
- all required source layers in the Agent Kit
- the extraction pass itself
- `source_projection_map`

- produced projection artifacts
- extraction-gap analysis
- derived structural and file-volume implications

This means it is the final summary artifact of the extraction stage.

---

7. What this artifact produces

The `kit_extraction_report` should produce explicit answers for:

- kit identity
- extraction coverage by layer
- extracted file counts
- consumed file refs
- projection artifacts produced
- unresolved extraction gaps
- derived repo/file implications
- extraction pass/fail status
- build eligibility recommendation

This makes it the main build-entry proof artifact for kit-driven building.

---

8. Required artifact structure

At minimum, `kit_extraction_report` should include:

1. artifact identity

2. kit identity
3. extraction coverage summary
4. source-layer extraction results
5. produced projection artifact summary
6. derived build implications
7. extraction gap report
8. extraction result and build gate recommendation

That is the minimum serious structure.

---

9. Top-level identity fields

At minimum, the artifact should contain:

- `kit_extraction_report_id`
- `run_id`
- `kit_ref`
- `created_at`
- `updated_at`
- `status`

This keeps it aligned with the rest of the Build Projection Pack.

---

10. Kit identity section

The report should explicitly identify the kit being extracted.

At minimum:

- `kit_ref`
- `kit_manifest_ref`
- `kit_version`
- `kit_file_count`
- `kit_root_path` or equivalent

This matters because the report must prove which kit it is speaking about, not just “a kit.”

---

11. Extraction coverage summary

The report should summarize extraction at a high level.

At minimum:

- total kit files
- total extracted files
- total ignored files
- total required source layers
- total extracted source layers
- total failed or incomplete source layers

This is the first place where extraction depth becomes measurable.

---

12. Source-layer extraction results

Each build-relevant source layer should have an explicit extraction result.

At minimum, for each layer:

- `source_layer`
- `required`
- `source_refs`
- `extracted`
- `extracted_count`
- `gap_count`
- `notes`

Required layers should include:

- brief
- canonical
- work breakdown
- acceptance
- architecture
- implementation
- design
- data
- API
- security
- relevant ops/governance

This section is one of the most important parts of the report.

---

13. Produced projection artifact summary

The report should explicitly list which projection artifacts were produced.

At minimum:

- `source_projection_map`
- `module_inventory`
- `ui_inventory`
- `endpoint_inventory`
- `entity_inventory`
- `file_inventory`
- `requirement_trace_map`
- `verification_trace_map`
- `repo_blueprint_seed`

For each artifact, the report should indicate:

- produced or not
- artifact ref
- completeness status

This proves extraction actually turned into build-facing structure.

---

14. Derived build implications section

The report should summarize what the extraction stage was able to derive.

At minimum:

- derived subsystem count
- derived module count
- derived screen count
- derived endpoint count
- derived entity count
- derived file-group count
- derived expected total file count
- derived repo type
- derived repo shape

This matters because it exposes whether the build has enough structure to be serious.

---

15. File-volume implication rule

This section should explicitly include file-volume implications.

The report should state:

- expected total file count
- required file count
- required directory count
- file count by major layer if available

This is critical for your system because it is how you detect whether the build is still collapsing the kit into too few files.

---

16. Extraction gap report

The report must explicitly list what extraction gaps remain.

At minimum, each gap entry should include:

- `gap_id`
- `source_layer`
- `source_ref`
- `gap_type`
- `severity`
- `description`
- `blocks_build`

Suggested `gap_type` values:

- `not_extracted`
- `partially_extracted`
- `projection_not_derived`
- `trace_not_formed`
- `file_implication_missing`

Suggested `severity` values:

- `warning`
- `error`

This prevents shallow extraction from being hidden.

---

17. Build gate recommendation

The report should explicitly state whether extraction is sufficient for downstream planning/build.

At minimum:

- `extraction_result`
- `build_gate_recommendation`

Suggested `extraction_result` values:

- `passed`
- `failed`
- `partial`

Suggested `build_gate_recommendation` values:

- `allow_build`
- `block_build`
- `allow_with_warnings` *(only if you explicitly support this policy)*

For a stricter first version, prefer:

- passed → allow_build
- failed/partial → block_build

---

18. Example layer result entry

{
 "source_layer": "architecture",
 "required": true,
 "source_refs": [
   "ARCH-01-system-boundaries.md",
   "ARCH-02-module-map.md",
   "ARCH-03-service-communication.md"
 ],
 "extracted": true,
 "extracted_count": 3,
 "gap_count": 0,
 "notes": null
}

---

19. Example projection artifact summary entry

{
 "artifact_name": "module_inventory",
 "produced": true,
 "artifact_ref": "projection/module_inventory.json",
 "completeness_status": "complete"
}

---

20. Example gap entry

{
 "gap_id": "GAP-003",
 "source_layer": "security",
 "source_ref": "SEC-03-data-protection.md",
 "gap_type": "projection_not_derived",
 "severity": "error",
 "description": "Security data-protection requirements were extracted but not projected into file or interface implications.",
 "blocks_build": true
}

---

21. Example report structure

A simplified example:

{
 "kit_extraction_report_id": "KER-0001",
 "run_id": "RUN-0001",

"kit_ref": "KIT-0001",
"kit_manifest_ref": "kit_manifest.json",
"kit_version": "1.0.0",
"kit_file_count": 309,
"kit_root_path": "kits/Atizay_kit",
"extraction_coverage_summary": {
  "total_kit_files": 309,
  "total_extracted_files": 287,
  "total_ignored_files": 22,
  "required_source_layers": 11,
  "extracted_source_layers": 11,
  "failed_source_layers": 0
},
"source_layer_results": [
  {
    "source_layer": "canonical",
    "required": true,
    "source_refs": ["canonical_spec.json"],
    "extracted": true,
    "extracted_count": 1,
    "gap_count": 0,
    "notes": null
  }
],
"projection_artifact_summary": [
  {
    "artifact_name": "module_inventory",
    "produced": true,
    "artifact_ref": "projection/module_inventory.json",
    "completeness_status": "complete"
  },
  {
    "artifact_name": "file_inventory",
    "produced": true,
    "artifact_ref": "projection/file_inventory.json",
    "completeness_status": "complete"
  }
],
"derived_build_implications": {
  "derived_subsystem_count": 8,
  "derived_module_count": 31,
  "derived_screen_count": 18,
  "derived_endpoint_count": 27,
  "derived_entity_count": 14,

```
  "derived_file_group_count": 22,
  "derived_expected_total_file_count": 720,
  "derived_repo_type": "fullstack_monorepo",
  "derived_repo_shape": "monorepo_with_shared"
 },
 "extraction_gaps": [],
 "extraction_result": "passed",
 "build_gate_recommendation": "allow_build",
 "created_at": "2026-03-06T12:02:00Z",
 "updated_at": "2026-03-06T12:02:00Z",
 "status": "active"
}
```


---

22. Anti-drift rules

`kit_extraction_report` should enforce these rules:

22.1 No invisible non-extraction

Required source layers cannot silently go unextracted.

22.2 No projection-free extraction claims

The system cannot claim successful extraction if required projection artifacts were not produced.

22.3 No build without structure

If extraction did not produce real structural implications, the build should not continue.

22.4 No file-count blindness

The report must expose derived file-volume implications.

22.5 No ignored gap suppression

Blocking extraction gaps must remain visible and build-relevant.

These rules directly protect your system from the shallow-kit-consumption problem.

---

23. Relationship to Build Mode gating

This artifact should feed a hard gate like:

G_BUILD_KIT_EXTRACTION

Pass condition:

- required source layers extracted
- required projection artifacts produced
- required structural implications derived
- no blocking extraction gaps
- build gate recommendation = allow_build

If this report does not pass, Build Mode should not proceed to blueprinting/planning/generation.

---

24. Relationship to Repo Blueprint and Build Plan

This is the main sequencing rule:

- `kit_extraction_report` proves extraction happened
- `repo_blueprint_seed` proves structural projection happened
- `repo_blueprint` elaborates that structure
- `build_plan` defines execution over it

So `kit_extraction_report` is the gate artifact that justifies downstream build artifacts.

---

25. Completion rule

`kit_extraction_report` should only be considered complete when:

- all required source layers have explicit extraction results
- all required projection artifacts have explicit produced/not-produced results

- structural and file-volume implications are visible
- extraction gaps are visible
- the report can make a real allow/block recommendation for build continuation

If it cannot do that, it is incomplete.

---

26. Success definition

BPP-12 is successful when `kit_extraction_report` is defined as a structured proof artifact that measures Agent Kit extraction coverage, produced projection artifacts, derived structural and file-volume implications, unresolved extraction gaps, and build-gate readiness, so the internal build system cannot proceed on shallow or unproven kit consumption.

---

27. Short version for the doc

BPP-12 — Kit Extraction Report
`kit_extraction_report` is the Build Projection artifact that proves whether the Agent Kit was extracted deeply enough to support a real build. It records source-layer extraction coverage, produced projection artifacts, derived structural and file-volume implications, unresolved extraction gaps, and the allow/block recommendation for downstream build execution. Its purpose is to stop Build Mode from starting on shallow kit usage.

Build Pipeline

Below is the **correct integration plan based on how your current repo already works**.

---

1. Where the Build Pipeline lives in the repo

Your current repo structure already follows a good pattern:

```
Axion/
├─ src/
│   ├─ stages/
│   ├─ mechanics/
│   ├─ verification/
│   ├─ types/
│   └─ ...
```

The **build pipeline should live beside the stage system**, not inside it.

Recommended location:

src/build/

Structure:

```
src/build/

 build_pipeline.ts
 build_runner.ts

 extractor/
   kit_loader.ts
   kit_extractor.ts

 projection/
   module_projector.ts
   ui_projector.ts
   endpoint_projector.ts
   entity_projector.ts
   file_projector.ts
   requirement_trace_projector.ts
   verification_trace_projector.ts
```

```
  repo_blueprint_seed_projector.ts

 blueprint/
  blueprint_deriver.ts

 planner/
  build_plan_generator.ts

 generator/
  repo_generator.ts
  file_generator.ts

 verification/
  build_verifier.ts

 export/
  zip_exporter.ts
```

This keeps build logic **isolated but native to Axion**.

---

2. Where build outputs live in runs

Your runs already look like this:

```
.axion/runs/RUN-0001/
  kit/
  planning/
  verification/
  state/
  run_manifest.json
```

The build pipeline should add:

```
.axion/runs/RUN-0001/build/
```

Inside that:

```
build/

 extraction/
  kit_extraction_report.json
```

```
projection/
  module_inventory.json
  ui_inventory.json
  endpoint_inventory.json
  entity_inventory.json
  file_inventory.json
  requirement_trace_map.json
  verification_trace_map.json
  repo_blueprint_seed.json

blueprint/
  repo_blueprint.json

plan/
  build_plan.json

repo/
  src/
  package.json
  ...

verification/
  build_verification_report.json

export/
  project_repo.zip
```

This keeps the build pipeline **fully attached to the run that produced the kit**.

---

3. When the build pipeline runs

The trigger condition should be:

```
run.stage == S10_CLOSE
AND kit.status == complete
```

So the flow becomes:

MAIN PIPELINE

S0 INIT

S1 INGEST
S2 VALIDATE
S3 STANDARDS
S4 CANONICAL
S5 TEMPLATE
S6 PLAN
S7 TEMPLATE_FILL
S8 GATE
S9 KIT_PACKAGE
S10 CLOSE

Then:

BUILD PIPELINE

B0 LOAD KIT
B1 KIT EXTRACTION
B2 BUILD PROJECTION
B3 BLUEPRINT DERIVATION
B4 BUILD PLAN
B5 CODE GENERATION
B6 BUILD VERIFICATION
B7 EXPORT REPO

The build pipeline is **not a stage** of the main pipeline.
It is a **separate runner** triggered afterward.

---

4. What the build runner actually does

The entry file should be:

src/build/build_runner.ts

Pseudo flow:

```
function runBuild(runId) {

 loadKit(runId)

 runKitExtraction()

 runProjection()
```

```
 deriveRepoBlueprint()

 generateBuildPlan()

 generateRepository()

 verifyBuild()

 exportZip()
}
```

Each step produces artifacts in:

.axion/runs/<RUN_ID>/build/

---

5. Where the Agent Kit is read from

The build pipeline **never touches the intake data**.

It reads from:

.axion/runs/<RUN_ID>/kit/

Which contains:

brief
canonical
planning
templates
implementation
acceptance
architecture
data
api
security

That is the **source of truth for Build Mode**.

---

6. Why this design is important

This structure keeps three layers clean:

Layer 1
MAIN PIPELINE
produces Agent Kit

Layer 2
BUILD PIPELINE
consumes Agent Kit

Layer 3
OUTPUT
Build Kit / Repo

That separation prevents:

- template generation leaking into repo generation
- build logic polluting the kit system
- the build agent improvising without projection artifacts

---

7. The first component you should implement

Do **not start with repo generation**.

Start with:

kit_extractor.ts

Goal:

kit → kit_extraction_report.json

If the extractor cannot fully extract the kit, the build pipeline stops.

This is the **safety system** that prevents shallow builds.

---

8. After that, implement projection

The next system is:

projection/

Which produces:

module_inventory
ui_inventory
endpoint_inventory
entity_inventory
file_inventory
requirement_trace_map
verification_trace_map
repo_blueprint_seed

These artifacts are what make **large builds possible (500+ files)**.

---

9. One critical thing for your system

Your build generator **must generate from `file_inventory`**.

Not from:

modules
screens
features

But from:

file targets

That is the key to solving the **200-file collapse problem** you found earlier.

---

10. What we should do next (important)

Before writing build code, the **next step should be locking the runtime artifact schemas** for:

kit_extraction_report
module_inventory
ui_inventory
endpoint_inventory
entity_inventory
file_inventory
requirement_trace_map
verification_trace_map
repo_blueprint_seed

These schemas become the **contracts the build engine must produce**.

Once those exist, building the engine becomes straightforward.

If you want, the **next thing I can do is map the Build Pipeline directly into your uploaded repo**, meaning I'll show you exactly:

which files to add
which files to modify
which folders to create